



MPDA: a data augmentation approach to improve deep learning for software vulnerability detection

Feiqiao Mao¹ · Yingxiang Yuan¹ · Xingyang Du¹ · Li Gao¹ · Zhihua Du¹

Accepted: 1 July 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

In the past few years, the advancement of deep learning based AI technologies has accelerated the research on automatic software vulnerability detection. However, deep learning models trained with vulnerability data have limited learning ability with high rates of false negative (FN) and false positive (FP), which are mainly caused by small and imbalanced sample problems of vulnerability data. Therefore, we propose a multiperspective data augmentation approach (called MPDA) and apply it to enhance data quality to improve deep learning for software vulnerability detection. MPDA automatically augments software vulnerability data from different perspectives by its three augmenting components designed as *augmenting by oversampling*, *augmenting by GAN*, and *augmenting by fuzzy sampling*. We also design three algorithms, the Oversampling Strategy Selection (OSS) algorithm, the GAN-based data generating algorithm, and the Fuzzy Sampling Strategy Selection (FSS) algorithm, to help MPDA automatically achieve the optimal augmentation effect. The evaluation results on the Juliet Java Suite dataset and the application of MPDA to five widely used models in deep learning-based vulnerability detection to detect 29 types of vulnerability demonstrated that our approach consistently improved the performance of each deep learning model for vulnerability detection by at least 12% in terms of the F1 score, except for Transformer, which is 3.9%.

Keywords Deep learning · Data augmentation · Software vulnerability detection · Data imbalance

1 Introduction

Software vulnerabilities pose a severe challenge in the realm of software security. These vulnerabilities can cause system instability, leakage of sensitive information, and may even be exploited by malicious actors for attacks. Conducting vulnerability detection can improve the quality and credibility of the software, thereby reducing the occurrence of software secu-

Communicated by: Tegawendé F. Bissyandé.

Extended author information available on the last page of the article

rity incidents. Effective software vulnerability detection methods can offer valuable references and practical tools for software developers and security researchers. Current research on software vulnerability detection mainly aims at improving automatic technologies to identify potential security vulnerabilities and weaknesses in the source code of software, aiming to enhance software security and stability. The National Vulnerability Database (NVD) published by the National Institute of Standards and Technology (NIST) has shown consistent increase in vulnerabilities from 2000 to 2022 (Byers et al. 2022), even reaching approximately 26,000 last year. These statistics clearly reveal the seriousness of the vulnerability problem and its continuous growth trend. Detecting software vulnerabilities is really time-consuming and a great challenge. The increasing number of vulnerabilities over the years has highlighted the need for intelligent and accurate vulnerability identification and attracted numerous software security researchers.

Over the years, researchers have proposed various methods and tools. They can be classified into static and dynamic analysis. Static analysis methods inspect vulnerabilities from the source code of software that is not running but often suffers from higher FN and FP rates (Austin et al. 2013). However, dynamic analysis methods need to run the software. With the rapid development of deep learning technologies, many researchers have tried to use them in software vulnerability detection (Saccante et al. 2019; Partenza et al. 2021; Wang et al. 2020, 2023; Li et al. 2018). Deep learning technologies have shown excellent performance in software vulnerability detection (Li et al. 2018, 2022; Saccante et al. 2019). However, the problems of small samples and data imbalance are significant challenges for improving the performance of vulnerability detection based on deep learning. A relatively small sample size or an insufficient number of positive samples restricts the learning performance of deep neural networks and can even prevent them from learning the characteristics of vulnerabilities (Yedida and Menzies 2022b). Moreover, the high computational costs in terms of computing power and time also pose a major challenge to the practicality of deep learning-based vulnerability detection.

As they found that the standard random oversampling method was not sufficient to solve the problem, Yedida et al. were the first to utilize improved oversampling techniques to augment software defect datasets and code smell datasets, respectively, with the aim of improving the performance of deep learning models in performing defect prediction tasks and code smell detection tasks (Yedida and Menzies 2022b, a). However, these datasets are composed of samples that describe the static attribute features of classes in projects with defects or code smells, rather than source code data samples. In contrast, the publicly available NIST vulnerability datasets are composed of the source code of classes with vulnerabilities. According to our literature survey, currently there is no research work that specifically focuses on improving vulnerability datasets and is used to improve the performance of deep learning models to which to refer.

In replication experiments of outstanding research work (Saccante et al. 2019) and initial exploratory experiments, we found and identified the issue of imbalance in software vulnerability datasets, characterized by substantial variance in sample numbers for different types of vulnerability. This data imbalance resulted in poor performance of deep learning models in detecting vulnerabilities in minority classes. Furthermore, insufficient publicly available vulnerability datasets limited training and evaluating deep learning models (Saccante et al. 2019; Shiri Harzevili et al. 2024; Croft et al. 2023). For example, in the NIST Juliet 1.2 Java test suite dataset, there are 528 samples in the CWE259 vulnerability dataset, while the

CWE191 vulnerability dataset contains 27,820 samples, showing a huge disparity between them. Multiple deep learning models such as Long Short Term Memory (LSTM) and Bidirectional Gated Recurrent Unit (BiGRU) generally have poorer F1 performance in detecting CWE259 compared to detecting CWE191. Moreover, for some types of vulnerability, the differences between positive and negative samples are so significant that the models cannot even detect them. For example, in the CWE563 vulnerability dataset, there are 218 positive samples and 716 negative samples; in the CWE398 vulnerability dataset, there are 137 positive samples and 378 negative samples. As a result, the F1 scores of LSTM and GRU in detecting CWE563 and CWE398 are directly zero. Moreover, in exploratory experiments, we also found that the oversampling algorithms used to achieve the best data augmentation effects for different types of vulnerabilities are not exactly the same.

Inspired by the work of Yedida and Menzies (2022b, 2022a), who utilized oversampling techniques to augment software defect datasets and software code smell datasets, aiming to improve the performance of deep learning models in performing software defect prediction tasks and code smell detection tasks, and motivated by the phenomena we observed in the initial exploratory experiments, we propose MPDA, which is a novel multiperspective data augmentation approach and applies it to enhance the ability of deep learning models to detect software vulnerability, in response to the challenges of small and imbalance vulnerability data samples. Specifically, the Juliet Java Suite dataset published by NIST is selected as our experimental benchmark that has also been used in the reference (Saccente et al. 2019). MPDA transformed and expanded vulnerability data from various perspectives to increase both diversity and quantity, thus improving data quality to better train deep learning models for software vulnerability detection. We applied more than four types of deep learning models to perform detection on augmented vulnerability samples. The experimental results unequivocally indicate that our MPDA approach is efficient and significantly improves the F1 performance of each deep learning model for the detection of vulnerabilities by at least 12%, except for Transformer, which is 3.9%.

The main work and contributions of this paper are as follows.

1. A new data augmentation approach called MPDA is proposed to improve deep learning for software vulnerability detection. The MPDA promotes data quality in enhancing the quantity and diversity by transforming and expanding software vulnerability samples from multiple perspectives.
2. Three algorithms which were oversampling-strategy-selection (OSS) algorithm, GAN-based data generating algorithm and fuzzy-sampling-strategy selection (FSS) algorithm were designed for MPDA to achieve the optimal augmentation effect.
3. The MPDA was verified in applying it to improve four widely used deep learning models for software vulnerability detection on the Juliet Java Suite dataset, demonstrating consistent effectiveness in promising improvement of each model.
4. The advancement of the MPDA was verified in improving the Bidirectional Long Short-Term Memory (BiLSTM) for software vulnerability detection against the commonly used benchmark known as Achilles on 29 types of vulnerabilities, demonstrating superior performance.

Paper Organization The rest six sections are organized as follows. Section 2 presents the background knowledge and foundations of our research. Section 3 is about the details of the

MPDA. The experimental setup and analysis of the results are explained in Section 4. Section 5 discusses validity threats from three aspects. Some relevant works are presented and reviewed in Section 6. Section 7 finally concludes the research.

2 Background

2.1 Software Vulnerability Detection Based on Deep Learning

Deep learning models such as LSTM (Du et al. 2017; Feng et al. 2020), GRU (Shen et al. 2018), BiLSTM (Pradhan et al. 2021; Kota and Munisamy 2022) and BiGRU (Zhang et al. 2018; Li et al. 2022), which have been widely used recently, often refer to artificial neural networks. Software vulnerability detection based on deep learning is a new way recently. It learns program features with neural networks from known vulnerabilities and then predicts vulnerabilities in software.

Deep learning-based vulnerability detection has made significant progress. For example, Li et al. (2018) initiated the study of deep learning-based vulnerability detection and proposed the first deep learning-based vulnerability detection system, VulDeepecker, which uses the BiLSTM neural network and detects vulnerabilities at the slice level considering semantic and syntax information. VulDeepecker was trained and evaluated on a dataset comprising samples of C/C++ code gadgets of two types of vulnerabilities: Common Weakness Enumeration (CWE)119 (Buffer Overflow) and CWE399 (Resource Management Errors). Later, they integrated these findings to develop the first systematic SySeVR framework for using deep learning to detect vulnerabilities in C/C++ programs with source code (Li et al. 2022), achieving an impressive F1 score of 92.6 with Bidirectional Gated Recurrent Unit (BiGRU) in vulnerability detection evaluated on their collected dataset which contains 126 types of vulnerabilities. Furthermore, the Achilles proposed by Saccente et al. (2019) achieved an accuracy greater than 90% for 24 out of 29 different types of CWE vulnerabilities evaluated on the Juliet 1.2 Java test suite dataset. Moreover, the surveyed literature (Liang et al. 2025; Semasaba et al. 2020; Shiri Harzevili et al. 2024) show that LSTM, GRU, BiLSTM, and BiGRU are commonly used deep learning network models in existing deep learning-based vulnerability detection methods.

Deep learning-based vulnerability detection methods have not only improved the automation and intelligence of vulnerability detection, but also contributed to enhancing accuracy and efficiency. With the ongoing evolution of deep learning technology, we believe that deep learning-based vulnerability detection will continue to advance software security.

2.2 Oversampling

Oversampling is a kind of data augmentation technology commonly used in the field of image processing, which balances datasets by increasing samples of the minority classes. There are several widely used oversampling algorithms, such as random oversampling, SMOTE proposed by Chawla et al. (2002), Borderline-SMOTE proposed by Han et al. (2005), and ADASYN proposed by He et al. (2008).

SMOTE is an improvement over random sampling, which synthesizes new samples by interpolating K nearest neighbors between samples from the minority class. Although

SMOTE performs well in interpolating simple minority-class samples, its effectiveness may diminish when dealing with complex minority-class distributions or noisy data.

ADASYN is an improvement over SMOTE. Depending on relative contributions, it determines how many synthetic samples should be generated for each minority class. ADASYN mainly uses the density distribution criterion to generate more synthetic samples in areas with low density. It better balances the datasets and produces synthetic samples that more accurately reflect the classification characteristics of the entire minority class. However, in extremely imbalanced scenarios, it may still face certain challenges.

As an improvement based on SMOTE, Borderline-SMOTE is specifically designed for oversampling samples that are close to the decision boundary. First, it identifies the boundary samples surrounding the minority class. Then, during the process of generating synthetic samples, it selectively uses these boundary samples as reference samples for synthesis instead of randomly selecting all neighboring samples. So Borderline-SMOTE focuses more on those samples that may be more representative and closer to the decision boundary.

Oversampling techniques are among the important means of resolving the problem of imbalanced datasets. When selecting an oversampling method, it is essential to make a prudent choice based on specific problems and perform appropriate parameter tuning to avoid introducing noise or resulting in overfitting. Some successful applications of oversampling techniques in deep learning-based software engineering tasks have already demonstrated their potential. The research by Yedida and Menzies (2022b) indicates that oversampling is effective and necessary before applying deep learning to the prediction of defects. They proposed a novel oversampling technique combined with fuzzy sampling to preprocess software defect data, helping deep learning models performing defect prediction tasks produce state-of-the-art results more quickly. However, what the dataset contains is not the class code of each software project with defects, but rather 20 static attributes of the classes, such as Weighted Methods per Class (WMC), Depth of Inheritance Tree (DIT), Number of Children (NOC), etc. Another research effort conducted by Yedida and Menzies (2022a) demonstrated that oversampling is also effective and necessary before applying deep learning to the detection of code smell. Although both code smell datasets and defect datasets suffer from the problem of sample imbalance (with a relatively small number of positive samples), the data that require preprocessing are not the samples containing the source code of classes, but rather the samples that describe the static attribute features of classes. This is completely different from software vulnerability datasets composed of the source code of classes. In addition, there are differences in the distributions of the feature vectors of different types of vulnerability. However, different oversampling algorithms are suitable for different domains and data characteristics, respectively. When dealing with vulnerability data, we will face another challenge that a single oversampling algorithm may not achieve the optimal data augmentation effect for vulnerability data. So, in the field of software vulnerability detection, it is imperative to consider the impacts of different oversampling algorithms on data augmentation and adopt optimal strategies to achieve superior detection results.

Consequently, in MPDA, we carefully select the most suitable oversampling algorithm from SMOTE, Borderline SMOTE, and ADASYN for each specific vulnerability type, in order to optimize the data augmentation method and increase the overall performance of the vulnerability detection models.

2.3 Generative Adversarial Network

GAN(Generative Adversarial Network) is recognized as one of the most promising deep learning approaches in unsupervised learning in complex distributions so far (Goodfellow et al. 2020). It is mainly composed of a generator and a discriminator. The generator consistently optimizes its generation capability with the objective of challenging the discriminator to differentiate generated data from real data. The discriminator continually improves the accuracy of its differentiation. These two components have conflicting objectives; they engage in an adversarial learning process.

The discriminator is a crucial component of GAN, and its main function is to classify input samples to distinguish between real and generated samples. Its objective is to accurately determine the source of the input sample and provide a probability estimate of its authenticity.

The generator is another essential component of GAN, and its main function is to take random noise as input and generate synthetic data which seem like real data in distribution characteristics. Its objective is to minimize the gap between the generated and real samples, making their distribution as close as possible.

GAN training is achieved through the alternation of generator and discriminator training. In each training iteration, a batch of real samples and a batch of random noise are randomly selected. The discriminator is trained using the real samples, while the generator is trained using the random noise. Through repeated iterations of this training process, the performance of the generator and discriminator gradually improves, resulting in more realistic generated samples.

GAN has demonstrated immense potential in various fields. It offers a novel approach to model building compared to traditional networks that rely solely on parameter updates. GAN utilizes adversarial learning to better approximate complex data distributions, making them more valuable for research purposes. Additionally, GAN learns features from unannotated data without the need for extensive labeling. So, it is good to generalize to new, unseen data.

And it has been widely applied in diverse fields, including, but not limited to image generation, image data augmentation, image restoration, image super-resolution, and image style transfer. It is capable of generating realistic synthetic samples, addressing the issue of limited data availability, and creating images with specific styles and characteristics, thus significantly advancing image processing and related domains. With the development of GAN technology, many researchers have explored its applications in natural language processing tasks, and thus achieved promising research results in data generation. Software vulnerability datasets are in the form of programming-language texts, which are similar to natural-language texts. So, GAN can explore its application in enhancing software vulnerability data.

2.4 Fuzzy Sampling

As another commonly used data augmentation technology in image processing, fuzzy sampling (Yedida and Menzies 2022a) plays a pivotal role in addressing the issues of imbalance and insufficient data. When the dataset is limited, fuzzy sampling generates additional synthetic samples to increase the data volume significantly, and thus improves the classifier's

performance. The core idea of this technology lies in the transformation and expansion of original samples to generate synthetic samples with slight variations. The generation of these synthetic samples can be based on random sampling and transformation technologies, such as adding random noise, applying fuzzy logical operations, and adjusting sample attributes. In research work on software code smell detection, fuzzy sampling has been used to enhance code smell datasets (Yedida and Menzies 2022a). So, fuzzy sampling can also be explored for its application in enhancing software vulnerability data.

3 The Proposed MPDA Approach

3.1 Overview of MPDA

Our MPDA approach aims to improve vulnerability data from different perspectives. Figure 1 illustrates the processes and components of the MPDA approach.

Firstly, MPDA takes Java software vulnerability data (a set of Java class files) as input and changes it into a vector matrix format for augmentation. Considering that the detection

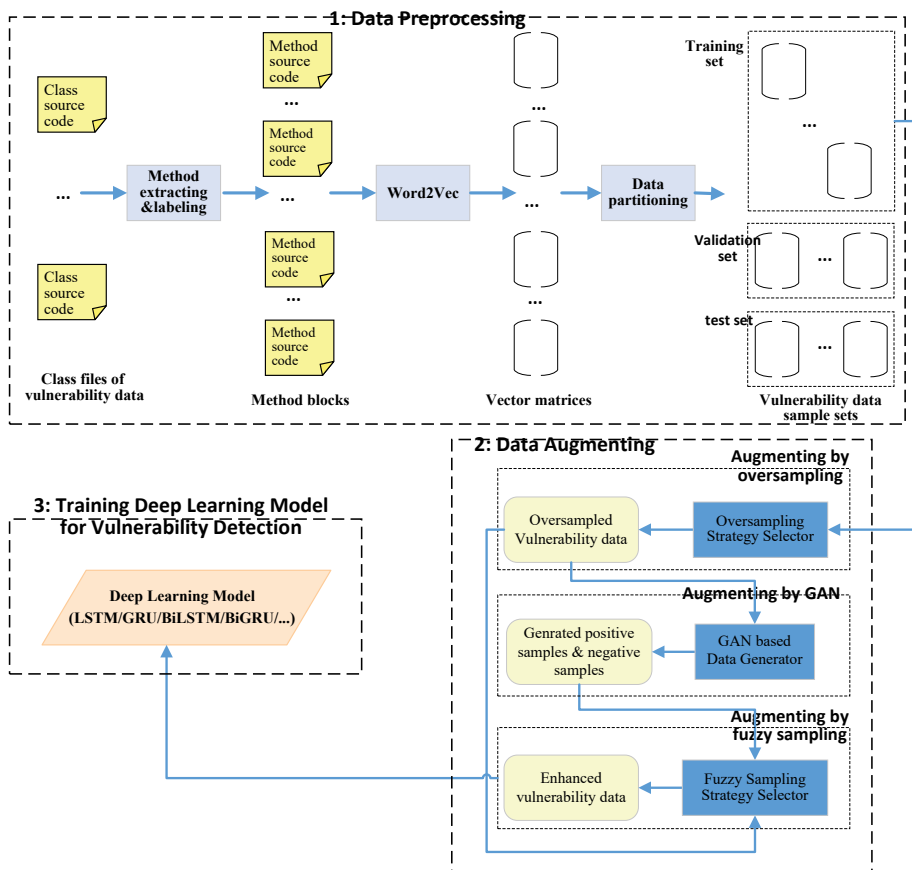


Fig. 1 General framework of the proposed MPDA approach

granularity is at the method level, where a single Java class may contain multiple method segments, MPDA utilizes a method extraction and tagging algorithm (Saccante et al. 2019) to transform the Java class source code dataset into method blocks. Subsequently, it uses Word2Vec proposed by Mikolov et al. (2013) to convert the method blocks into vector matrices. To ensure the consistency and authenticity of the detection results, transformed vector format data is partitioned into training, validation, and testing sets with the ratio of 8:1:1 as conventional. The training set is input into the subsequent data augmentation process and output as augmented dataset to train the downstream deep learning models. The validation set is used to evaluate the trained parameters and not used to tuning hyperparameter. The test set is used to evaluate the model's performance in vulnerability detection. With the *data preprocessing* process, MPDA converts and segregates the vulnerability datasets from text format to different types of vector format, thus preparing for the following augmenting process and the training deep learning model process for vulnerability detection.

Secondly, MPDA increases the diversity and quantity of vulnerability data with its three augmenting components, including *augmenting by oversampling*, *augmenting by GAN* and *augmenting by fuzzy sampling*. We designed the OSS algorithm for the component of *augmenting by oversampling* to better adapt to different types of vulnerability. The *augmenting by oversampling* process mainly addresses data imbalance by generating similar samples of the minority class to achieve a balanced distribution of vulnerability datasets. The process of augmenting by GAN mainly addresses the issue of insufficient data in publicly available vulnerability datasets by generating synthetic vulnerability data with realistic characteristics, in which the GAN based generator generates positive and negative samples, respectively, thereby increasing the size of the dataset and providing more diverse samples. We designed the FSS algorithm for the component of *augmenting by fuzzy sampling*, providing four strategies for various vulnerabilities and downstream neural network models. The process of augmenting by fuzzy sampling mainly addresses both issues of imbalance and insufficient data by introducing data perturbations on different occasions to produce the enhanced data. During the data augmentation process, both the characteristics of the vulnerability data and the domain knowledge are taken into account to ensure that the augmented data remain interpretable and accurate.

The final process of MPDA is to train the deep learning model for vulnerability detection. The augmented vulnerability dataset is input into the deep learning model to learn vulnerability detection. The deep learning model can be LSTM, GRU, BiLSTM, BiGRU, etc.

In order to effectively enhance the quantity of vulnerability data, our MPDA approach augments data from multiple perspectives, thus improving the vulnerability detection performance of deep learning network models. Additionally, there are several practices of the proposed MPDA that require special explanations as follows.

We chose Word2Vec for code representation and embedding mainly because it has been adopted in numerous research works on deep learning-based vulnerability detection (Shiri Harzevili et al. 2024), such as Tang et al. (2022), Li et al. (2022), and Wang et al. (2023), etc. Moreover, the Achilles vulnerability detection method (Saccante et al. 2019), with which MPDA is directly compared in this paper, uses the same dataset as ours and also uses Word2Vec.

Besides, during the data preprocessing stage of the MPDA method, we considered several options, including Code2Vec, CodeBERT, Word2Vec, Unixcoder, and CodeT5+. Eventually, we chose Word2Vec. The reason is that Code2Vec is based on the Abstract Syn-

tactic Tree (AST) paths of code and relies on manually designed AST path features. This approach has significant limitations in the preprocessing of data for large-scale and complex vulnerability datasets. Moreover, compared to CodeBERT, Unixcoder, and CodeT5+, the use of Word2Vec for code representation incurs the lowest computational cost in data preprocessing.

Moreover, at the end of the data preprocessing stage of the MPDA method, the data are divided into training, validation and test sets using an 8: 1: 1 ratio. We selected this division ratio primarily because it suits vulnerability datasets and can balance the requirements of model training and evaluation on our vulnerability datasets, preventing overfitting. The specific reasons are as follows. Typically, when the sample size is less than 10,000, the dataset is small or medium-scale. In such scenarios, the 8:1:1 division ratio has the advantage of providing adequate training data while retaining reliable validation/test samples. It is a benchmark method in both traditional machine learning and deep learning and is widely applied in academic research and industrial projects. Since vulnerability datasets are small or medium-sized with issues of imbalance and small sample size, this ratio is appropriate. Moreover, although the 6:2:2 division ratio is recommended in some cases, we did not adopt it mainly due to our focus on prioritizing training data. In small-scale data scenarios, the model performance is much more sensitive to the amount of training data than the validation/test data. The literature (Goodfellow et al. 2016) indicates that the amount of training data is the key determinant of model performance. Especially when data are limited, maximizing the proportion of the training set should be prioritized. Google Research (Sun et al. 2017) demonstrates that when the data volume is less than 10,000, increasing the proportion of the validation/test set beyond 10% has a minimal impact on improving evaluation results, yet reducing training data will significantly affect model performance. If we divide the dataset in a 6:2:2 ratio, given the small sample problem of vulnerability datasets, there will be fewer positive samples of the corresponding types of vulnerability and the model may fail to learn the key features of those types of vulnerability. Furthermore, according to the law of large numbers in statistical theory, the variance of evaluation indicators is inversely proportional to the sample size. For example, when the validation/test set grows from 100 samples (8:1:1) to 200 samples (6:2:2), the variance is only reduced by around 50%, but a 20% reduction in training data may lead to a significant increase in model bias.

3.2 Augmenting by Oversampling

During our exploratory experiments to choose the oversampling algorithm, we found that the distributions of the generated feature vectors for different types of vulnerability were distinct. Using a single oversampling algorithm may not yield the optimal data augmentation effect. Each of the above three algorithms has its own applicable domain. Such as the SMOTE algorithm is suitable for simple minority class sample interpolation and is more appropriate for datasets where minority class samples are relatively simple and linearly separable. Specifically, this algorithm is well suited for addressing vulnerabilities such as CWE113, CWE190, and CWE191. However, the ADASYN algorithm is suitable for highly imbalanced datasets and can intelligently adjust the number and distribution of synthetic samples, making it more adaptable to complex data distributions, and shows better performance when dealing with vulnerabilities such as CWE15, CWE36, and CWE398. Lastly, Borderline SMOTE can effectively increase the number of vulnerability samples near the

decision boundary, thus enhancing the classifier's ability to recognize boundary samples and improving the overall detection performance. However, Borderline SMOTE identifies boundary samples based on the data distribution itself rather than relying on a specific classifier. Even when different deep learning models are used in vulnerability detection tasks, the generated samples remain valid. It is particularly useful for complex datasets, focusing on samples near the decision boundary to improve the effectiveness of oversampling. It is particularly applicable to vulnerabilities such as CWE23, CWE129, and CWE134. Inspired by these observed experimental phenomena, we speculate that the characteristics of the data of certain types of vulnerability might share similar tendency patterns, which enables them to achieve the best data enhancement effect when using the same oversampling algorithm. In this case, such an oversampling algorithm would be the most suitable to oversampling the data for these types of vulnerability. According to this, we divide various types of vulnerability into three sets, which are SMOTE_fit VT_set, Borderline_SMOTE_fit VT_set, and ADASYN_fit VT_set, as shown in Table 1. It should be noted that the oversampling adaptation sets are divided on the basis of Word2Vec embeddings. If the embedding method is changed, relevant experiments must be performed anew. It is necessary to reevaluate the distribution characteristics and adjust the partitioning of the set.

In MPDA, the process of *augmenting by oversampling* mainly addresses the issue of unbalanced vulnerability data. We designed the OSS algorithm shown as Algorithm 1 and implemented it as the OSS component to augment the vulnerability data. In the process, we provide three different oversampling algorithms, such as SMOTE, ADASYN, and Borderline SMOTE, for different types of vulnerability. We also classify different vulnerability types into three sets as SMOTE_fit VT_set, Borderline_SMOTE_fit VT_set and ADASYN_fit VT_set. Therefore, we can flexibly adjust these algorithms according to different types of vulnerability to achieve a better augmentation effect. Notably, the three oversampling algorithms: SMOTE, ADASYN, and Borderline SMOTE, used in the implementation of our OSS algorithm, enrich and balance the vulnerability dataset by generating approximate samples of vulnerabilities of the minority class. Therefore, in MPDA, the process of *augmenting by oversampling* that is implemented as the OSS algorithm also primarily addresses the issue of data imbalance by generating similar samples of the minority class.

Algorithm 1 Oversampling Strategy Selection.

Input: Data to be oversampled D, Vulnerability types VT

Output: Oversampled data OD

1: Receive D and VT as input

2: **switch** VT

3: **Case** VT belongs to SMOTE_fit VT_set **then**

 Select SMOTE algorithm; **break**;

4: **Case** VT belongs to Borderline_SMOTE_fit VT_set **then**

 Select Borderline_SMOTE algorithm; **break**;

5: **Case** VT belongs to ADASYN_fit VT_set **then**

 Select ADASYN algorithm; **break**;

6: **Default:**

 Select ADASYN algorithm; **break**;

7: **End switch**

8: Apply the selected oversampling algorithm to D and generate synthetic samples to balance the class distribution;

9: Combine D with the synthetic samples as OD;

10: **Return** OD

Table 1 Sets of various types of vulnerability for oversampling strategy selection

VT set	Various types of Vulnerability
SMOTE_fit VT_set	CWE113, CWE190, CWE191, CWE197, CWE470, CWE476, CWE506, CWE606, CWE690, CWE80, CWE90
Borderline_SMOTE_fit VT_set	CWE129, CWE134, CWE23, CWE259, CWE369, CWE400, CWE601, CWE643, CWE81
ADASYN_fit VT_set	Vulnerability types that are neither in SMOTE_fit VT_set nor in Borderline_SMOTE_fit VT_set.

Note: VT set represents vulnerability type set

Since we have already validated known types of vulnerability (especially the 29 types of common vulnerabilities (Saccente et al. 2019) in the dataset of the NIST Juliet 1.2 Java test suite), we selected the best oversampling algorithm based on the discovery result of our exploratory experiment for them. However, during the experiment, we also found that the ADASYN algorithm consistently showed good oversampling performance. Therefore, for unknown and unverified types of vulnerability, we use the ADASYN algorithm.

By selecting the corresponding oversampling algorithm based on the type of vulnerability data, the OSS balances and enhances the diversity of vulnerability data. Moreover, this flexible selection strategy allows our approach to demonstrate better adaptability and generalization when automatically dealing with different types of vulnerability datasets.

3.3 Augmenting by GAN

As shown in Algorithm 2, we designed the GAN-based data generating algorithm to generate high-quality vulnerability samples in the process of *augmenting by GAN*. The oversampled vulnerability samples are divided into positive and negative. Then we use positive samples to train the pGAN model, while we use negative samples to train nGAN model. Through this training process, new positive and negative vulnerability data are generated that closely resemble real vulnerability data. This process helps to address the issue of insufficient data in publicly available vulnerability datasets and provide higher quality data to train downstream vulnerability detection models.

Algorithm 2 GAN based data generating.

Input: Oversampled data OD, batch size, input dim

Output: Generated data GD

- 1: Obtain positive samples from OD;
- 2: Obtain negative samples from OD;
- 3: Define the discriminator $D(X_{pSample})$ and $D(X_{nSample})$;
- 4: Define the generator $G_p(z)$ and $G_n(z)$;
- 5: Define the pGAN model with $G_p(z)$ and $D(X_{pSample})$;
- 6: Define the nGAN model with $G_n(z)$ and $D(X_{nSample})$;
- 7: Use the defined $D(X_{pSample})$, $G_p(z)$, positive samples, batch size and input dim to train the pGAN model;
- 8: Use the trained pGAN model to generate new positive samples;
- 9: Use the defined $D(X_{nSample})$, $G_n(z)$, negative samples, batch size and input dim to new train the nGAN model;
- 10: Use the new trained nGAN model to generate new negative samples;
- 11: Combine positive samples with negative samples as GD;
- 12: **Return** GD

The discriminator of pGAN (nGAN) is represented by the function $D_p(X_{pSample})$ (or $D_n(X_{nSample})$) as shown in (1), where $X_{pSample}$ is the input positive vulnerability sample, $X_{nSample}$ is the input negative vulnerability sample. W^D and b^D are the parameters used for the linear transformation and bias of the input $X_{pSample}(X_{nSample})$ sample. The sigmoid function is used as an activation function to constrain the output between 0 and 1 in the discriminator. This represents the discriminator's probability estimate of the input real samples.

$$D_p(X_{pSample}) \text{ or } D_n(X_{nSample}) = s(W^D x + b^D) \quad (1)$$

The generator of pGAN (nGAN) is represented by the function $G_p(z)$ (or $G_n(z)$) as shown in (2), where z is the input random noise. W^G is the parameter for the linear transformation. b^G is the parameter for the bias of the input random noise. In order to maintain the non-negativity of the output, the ReLU function is used as an activation function in the generator.

$$G_p(z) \text{ or } G_n(z) = g(W^G z + b^G) \quad (2)$$

The training process of pGAN is represented by (3), where the D_p represents the discriminator of pGAN, G_p represents the generator of pGAN, x represents real positive samples and satisfies the distribution of $P_{data}(x)$, z represents random noise and satisfies the distribution of $P_z(z)$. G_p and D_p engage in a game to maximize the objective function $V(D_p, G_p)$. That is, the discriminator D_p tries to accurately distinguish real positive samples from those generated by the generator G_p , while G_p strives to generate more realistic synthetic samples to deceive D_p . While the training process of nGAN is similar with pGAN as (3), but the x represents real negative samples, and then $P_{data}(x)$ represents the true negative data distribution. Furthermore, D_p is replaced by D_n , G_p is replaced by G_n .

$$\min_{D_p} \max_{G_p} V(D_p, G_p) = E_{x \sim P_{data}(x)} [\log D_p(X_{pSample})] + E_{z \sim P_z(z)} [\log (1 - D_p(G_p(z)))] \quad (3)$$

3.4 Augmenting by Fuzzy Sampling

Fuzzy sampling plays a critical role in data augmentation by introducing variations and diversity, effectively improving the richness of the dataset and improving the generalization ability of the model. As a result, even with limited data or imbalanced samples, the model can better adapt to complex data distributions and accurately classify and predict unseen data. Through well-designed sampling and transformation strategies, it can purposefully enhance the data, mitigating issues of over-fitting or under-fitting in the model, thus improving its robustness and reliability. In practical applications, researchers and engineers should flexibly choose fuzzy sampling technologies according to the characteristics of datasets in a specific task, to achieve optimal performance in tackling various complex problems.

In the process of augmenting by fuzzy sampling, adding Gaussian noise is used, which introduces a certain level of randomness and uncertainty to the original data. The fuzzy sampling process is represented by (4), where the x_{new} represents the processed vulnerability data, x represents the vulnerability data to be processed, ave represents the average value, and $sDIT$ represents the standard deviation. By adding a Gaussian noise, we can introduce

some randomness while preserving the main characteristics and structure of the vulnerability data. This technology allows us to generate more diverse samples, thus expanding the amount of vulnerability data and increasing the diversity of the dataset.

$$x_{new} = x + noise(ave, sDVI) \quad (4)$$

On the other hand, according to our initial exploratory experiments, we classify different types of vulnerability into four sets as FSStrategy_1_fit VT_set, FSStrategy_2_fit VT_set, FSStrategy_3_fit VT_set, and FSStrategy_4_fit VT_set. We also classify different deep learning models into two sets as basic NN_set and variant NN_set. For example, LSTM and GRU belong to the basic NN_set, while BiLSTM and BiGRU belong to the variant NN_set. The neural networks which are in the variant NN_set are more complex. Vulnerability data often exhibit complex features and distributions, and Gaussian noise enables us to simulate uncertainty and randomness present in the real world. Additionally, we can flexibly control the degree of fuzziness in the samples by adjusting the parameters of Gaussian noise, thus meeting various data augmentation requirements. This flexibility allows us to effectively enhance the quality and diversity of datasets, ultimately improving the performance of downstream vulnerability detection models.

The specific rationales underlying our approach are as follows. In our initial exploratory experiments on the selection of data augmentation techniques, it was found that different types of vulnerability data did not use exactly the same data augmentation strategies to achieve the best fuzzy sampling data augmentation effect. Therefore, based on the data augmentation effects of various types of vulnerability data in the exploratory experiments, we designed the four data augmentation strategies to allow 29 types of vulnerability data to achieve their respective optimal data augmentation effects. In addition, we divide the 29 vulnerability types into four sets adapted to optimal fuzzy sampling strategies, namely the FSStrategy_1_fit VT_set, the FSStrategy_2_fit VT_set, the FSStrategy_3_fit VT_set, and the FSStrategy_4_fit VT_set, as shown in Table 2, to choose the most appropriate strategy for the 29 known and verified vulnerabilities. However, when dealing with unknown or unverified vulnerabilities, we also discovered in the exploratory experiments that the choice of optimal strategy is closely related to the complexity of the downstream task models. Specifically, when downstream task models adopt relatively basic and simple deep learning models, such as LSTM and GRU, the second strategy can yield the best results. For more complex and advanced deep learning models, such as BiLSTM and BiGRU, since these models are more sensitive to contextual information and can better capture similar, yet diverse, data features, in this case, the third strategy is more effective and can achieve better results. Consequently, we further divide the deep learning models into two types of sets, namely the basic NN_set and the variant NN_set, as shown in Table 2, to determine data augmentation strategies when encountering other types of vulnerability other than the 29 types of vulnerability.

The four strategies we have devised are elaborated in the following.

Strategy 1, the first strategy involves combining the oversampled dataset with the GAN-generated data after applying fuzzy sampling, building a new training dataset.

Strategy 2, the second strategy combines the over-sampled dataset with the data generated by GAN and the over-sampled dataset after fuzzy sampling, creating a new dataset.

Strategy 3, building upon the second strategy, the third further applies fuzzy sampling to the data generated by GAN before combining them with the over-sampled dataset.

Strategy 4, the final strategy is to combine the dataset over-sampled with GAN-generated data and fuzzy sampled data.

To achieve the optimal effect for fuzzy sample acquisition data, we designed the FSS algorithm as shown in Algorithm 3 and provided the FSS component. For known and validated vulnerabilities, we choose the most suitable strategy for each specific vulnerability (lines 3-6). However, for unknown vulnerabilities or those not validated, our experimental experience is that the best strategy is decided by the complexity of the downstream models. In particular, when the downstream model is a relatively basic neural network, such as LSTM and GRU, the second strategy yields the best results (lines 7-8). However, for more complex neural network models like BiLSTM and BiGRU, these models are more sensitive to contextual information and can better capture features from similar but diverse data. In such cases, the third strategy is more effective in achieving better results (lines 9-10).

Algorithm 3 Fuzzy sampling strategy selection.

Input: Data to be fuzzy sampled D, Vulnerability type VT, Neural network type NNT

Output: Enhanced data ED

```

1: Receive D, VT and NNT as inputs;
2: switch VT belongs to:
3:   Case VT belongs FSSstrategy_1_fit VT_set then
     Select Strategy 1; break;
4:   Case VT belongs FSSstrategy_2_fit VT_set then
     Select Strategy 2; break;
5:   Case VT belongs FSSstrategy_3_fit VT_set then
     Select Strategy 3; break;
6:   Case VT belongs FSSstrategy_4_fit VT_set then
     Select Strategy 4; break;
7:   Default:
8:     if NNT belongs to basic NN_set then
       Select Strategy 2;
9:     else if NNT belongs to variant NN_set then
       Select Strategy 3;
10:    end if; break;
11: end switch
12: Generate composite samples on D with the selected strategy;
13: Add the composite samples to ED;
14: Return ED

```

For simpler models, we prioritize providing more realistic and diverse data samples to enhance the data set and avoid overfitting. In contrast, for more complex models, we further enhance the diversity of data to better capture the characteristics of vulnerability data. Our goal is to maximize the impact of data augmentation in the vulnerability detection task, helping the deep learning model to better adapt to different vulnerability scenarios and contextual information, thus improving its accuracy and reliability.

By employing such strategy selection, we can fully leverage oversampling, GAN, and fuzzy sampling techniques for different types of vulnerability and various deep learning models. This enables us to create more representative and diverse data sets, thus providing better support for the training and performance of vulnerability detection models.

Table 2 Sets of various types of vulnerability and deep learning models for the FSS component

VT sets	Various types of Vulnerability
FSSstrategy_1_fit VT_set	CWE197, CWE789, CWE81
FSSstrategy_2_fit VT_set	CWE129, CWE15, CWE190, CWE23, CWE259, CWE319, CWE369, CWE563, CWE606, CWE690, CWE78
FSSstrategy_3_fit VT_set	CWE113, CWE134, CWE191, CWE506, CWE601, CWE83, CWE89, CWE90
FSSstrategy_4_fit VT_set	CWE36, CWE398, CWE400, CWE470, CWE476, CWE643, CWE80
DL set	Deep learning model
basic NN_set	LSTM, GRU, CNN
variant NN_set	BiLSTM, BiGRU, Transformer, etc.

Note: VT set represents vulnerability type set. DL set represents deep learning models set

4 Experimental Evaluations

4.1 Experimental Setup

4.1.1 Datasets

The dataset used in our experiment was the Juliet 1.2 Java test suite, downloaded from the Open Web Application Security Project (OWASP) community's repository on GitHub¹. It contains examples for 112 different CWEs. We did not directly use the latest version, Juliet 1.3, released on the official NIST website in 2017 for the following reasons.

Firstly, the OWASP community is a non-profit organization with significant influence in the field of software security, has its shared security knowledge, practical experience and standards that receive wide attention and recommendations from numerous companies, governments, organizations, security professionals, and developers worldwide. Secondly, its Juliet Java test suite resource package is taken from the official NIST website². Lastly, compared to Juliet 1.2 released in 2012, the main changes in Juliet 1.3 on the NIST official website lie in the C cases of the Juliet C/C++ test suite. Regarding the Juliet Java test suite that we used, only minor changes were made, mainly the addition of integer overflow cases involving a prefix increment (++i) or postfix increment (i++) operator under CWE190 in s06 and s07, and of underflow cases involving a prefix decrement (–i) or postfix decrement (i–) operator under CWE191 in s04 and s05. 851 cases (1403 files) were added in each new Java subdirectory, for a total of 3404 Java cases (5612 files), which hardly affects our verification experiment.

We mainly used the 29 types of common and important vulnerability in the Juliet 1.2 Java test suite, which are shown in Table 3. Table 3 presents the number of class files, positive samples, negative samples, and the total number of samples included in the dataset for each type of vulnerability.

¹<https://github.com/find-sec-bugs/juliet-test-suite>

²<https://samate.nist.gov/SARD/test-suites>

Table 3 Number of samples of the 29 types of common vulnerability in the raw dataset

Vulnerability type	Num of class files	Num of samples		
		<i>P</i>	<i>N</i>	<i>Total sample</i>
CWE190	7015	6820	21000	27820
CWE191	5612	5456	16800	22256
CWE129	4392	4197	12873	17070
CWE89	3660	3465	10605	14070
CWE369	3050	2890	8862	11752
CWE789	2537	2456	4601	7057
CWE400	2396	2279	6977	9256
CWE113	2196	2079	6363	8442
CWE197	1980	1890	3519	5409
CWE134	1098	1020	3108	4128
CWE80	1080	1016	1886	2902
CWE643	732	237	639	876
CWE606	732	693	2121	2814
CWE36	720	691	1288	1979
CWE90	720	248	391	639
CWE15	720	691	1288	1979
CWE23	720	691	1288	1979
CWE470	720	691	1288	1979
CWE78	720	691	1288	1979
CWE319	610	493	1449	1942
CWE81	540	508	943	1451
CWE601	540	508	943	1451
CWE83	540	508	943	1451
CWE690	489	488	1512	2000
CWE476	293	293	892	1185
CWE563	250	218	716	934
CWE259	180	183	345	528
CWE398	137	137	378	515
CWE506	116	116	143	259
Total	44495	41653	114449	156102

Note: *P* represents the number of positive samples. *N* represents the number of negative sample. A positive samples is a vulnerability sample, while a negative sample is not

4.1.2 Baselines

To fully verify the effectiveness and superiority of the MPDA method in improving the performance of deep learning models in vulnerability detection task, the baseline models we used include GRU, LSTM, BiGRU, BiLSTM and Transformer. In evaluating the effect of the proposed MPDA, we applied it to four deep learning models: LSTM, GRU, BiLSTM and BiGRU, and a currently popular one that is Transformer, in vulnerability detection. We compared our method with other data augmentation methods in advancing different deep learning models performing vulnerability detection task. The main reason why we chose these models is that they are the deep neural network models commonly used in current deep learning-based vulnerability detection methods (Liang et al. 2025; Semasaba et al. 2020; Shiri Harzevili et al. 2024; Li et al. 2018), especially those with excellent perfor-

mance (Li et al. 2022; Saccente et al. 2019). In addition, BiLSTM is the most frequently used model, appearing in 20 studies. GRU and LSTM are also popular models with 14 and 13 studies, respectively (Shiri Harzevili et al. 2024).

Furthermore, given that the Transformer boasts advanced performance and has been a highly popular deep neural network model in recent years, being prevalently adopted in standard general LLM (Large Language Models) and codeLLMs. Hence, we have also evaluated the effectiveness of MPDA in improving the vulnerability detection performance of the Transformer.

The data augmentation baseline methods we used are SMOTE oversampling, GAN, and fuzzy sampling methods.

The deep learning-based vulnerability detection method adopted is the Achilles method (Saccente et al. 2019), as it is, to our knowledge, the most excellent approach to Java vulnerability detection and uses the same dataset as ours. We did not conduct a comparative evaluation with the two excellent deep learning-based vulnerability detection methods mentioned before, namely VulDeepecker and SySeVR, mainly for the following reasons. Firstly, both SySeVR and VulDeepecker are designed to detect vulnerabilities in C/C++ programs and are unable to handle Java programs. In contrast, the current implementation of MPDA is aimed at detecting vulnerabilities in Java programs, and it is still not capable of handling C/C++ programs yet. Secondly, VulDeepecker and SySeVR use specialized self-built datasets of C/C++ programs. Although these datasets have been publicly provided by the authors, they require specific data preprocessing procedures. However, MPDA uses the publicly available general Java vulnerability dataset from the NIST. Given that the ground truth is different, they cannot be compared. However, Achilles is the deep learning-based system that we know performs best in detecting vulnerabilities in Java programs, and it uses the same dataset as we do, which is the NIST Juliet 1.2 Java test suite. Therefore, when evaluating the advancement of the MPDA method in software vulnerability tasks, we chose to compare it with Achilles.

4.1.3 Hyper Parameters

Table 4 shows the values of certain typical hyper parameters configured in our experiment. For the GAN, the parameters that require setting are mainly the latent dimension (LATENT_DIM), the batch size (BATCH_SIZE_GAN), and the number of epochs (EPOCH_GAN). For the five deep learning models used (LSTM, GRU, BiLSTM, BiGRU, and Transformer), the parameters that require setting are the sequence length of input samples (MAX_LEN), the batch size (BATCH_SIZE_NET), the number of epochs (EPOCH_NET), the dropout rate (DROPOUT_RATE), the loss function (LOSS_FUNCTION), and the activation function (ACTIVATION_FUNCTION).

Hyperparameter tuning is a method to improve the performance of the model. However, our research focused on evaluating the performance improvement brought about by our proposed MPDA data augmentation method for deep learning models in vulnerability detection tasks. To minimize the interference caused by hyperparameter tuning in the performance improvement of the model as much as possible, we did not deliberately perform hyperparameter tuning for each model used in our experimental environment and dataset. Instead, we used the untrained and randomly initialized version of each model. During our experiments, to ensure a fair comparison, the typical hyperparameter settings are kept consistent

for each model. Moreover, in the experiment to verify the performance improvement of each model with the MPDA method over other data augmentation methods for the vulnerability detection task, we kept the hyperparameter settings of the same model consistent.

The experiments are implemented on an Inspur AI station server with Intel(R) Xeon(R) Gold 6226R CPU (2.90GHz, 2 cores), 503GB RAM, 2 NVIDIA RTX 3090 with 24GB memory, running Ubuntu 22.04. The software and libraries used include IDE VS Code 1.95.3, python 3.9.21, tensorflow-gpu 2.6.0, pytorch 2.5.1 and cuda12.1.

The specific procedures for the training and testing of each model are as follows. Firstly, select a model, for example, BiLSTM, and set its hyperparameters. Then, loop through the datasets of each of the 29 types of vulnerability in the vulnerability dataset. When traversing a specific type of vulnerability, such as CWE190, randomly divide the CWE190 vulnerability dataset into a training set, a validation set, and a test set in a ratio of 8: 1: 1. Randomly initialize the selected BiLSTM and feed the data from the CWE190 vulnerability training set into the randomly initialized BiLSTM for 20 training epochs. During the training process, at the end of each epoch, calculate the performance metrics and losses on the CWE190 validation set. If the loss decreases, retain the weight parameters and continue training and validation for the next epoch starting from these weight parameters. If the loss does not decrease, do not retain the weight parameters and continue training and validation for the next epoch starting from the weight parameters of the previous epoch. After the 20 epochs of training and validation have been completed, use the CWE190 test set to test and calculate the performance metrics of the BiLSTM in detecting the vulnerability CWE190. Training and testing processes for other types of vulnerability are carried out in the same manner. After the above loop traversal is completed, calculate the average performance metrics of the 29 types of vulnerability. This training and testing experimental process is repeated 10 times and the average value of the results of these 10 experiments is taken as the detection performance result of the BiLSTM model on the original vulnerability dataset and presented. The approach for training each model using the augmented vulnerability dataset is consistent with the above training and testing process on the raw vulnerability dataset. The only difference is that, during the training process, the raw vulnerability data training set is replaced with the augmented vulnerability data training set, while the validation set and the test set remain unchanged.

4.1.4 Evaluation Metrics

Four metrics of Accuracy, Precision, Recall, and F1 score metrics are widely used in the evaluation of software defect detection, such as Luo et al. (2022), Partenza et al. (2021),

Table 4 Hyper parameters used in the evaluation

Hyper parameters	Value	Hyper parameters	Value
LATENT_DIM	100	BATCH_SIZE_GAN	32
MAX_LEN	140	BATCH_SIZE_NET	128
LEARNING_RATE	0.001	EPOCH_GAN	100
DROPOUT_RATE	0.5	EPOCH_NET	20
ACTIVATION_FUNCTION	ReLU	LOSS_FUNCTION	Binary-Cross-entropy

Wang et al. (2020) and Zhang et al. (2021). So we also use them as our evaluation metrics in this paper.

4.2 Results and Analysis

4.2.1 Effect of MPDA in Data Augmentation

The samples of the training set of the datasets of 29 types of vulnerability before and after augmentation by MPDA is presented in Table 5. It can be seen that there were 124871 samples in total for the training sets of the datasets of 29 types of vulnerability before augmentation. After being augmented by MPDA, the number of samples increased to 392051, representing a 213.9% $((392051-124871)/124871*100\% = 213.9\%)$ increase in scale. This shows that MPDA is remarkably effective in increasing the scale of vulnerability data. There are 193157 positive samples and 198894 negative samples after augmentation, with the average of positive and negative samples being 196026. The negative deviation of positive samples from the mean is 1.46% $((196026-193157)/196026*100\% = 1.46\%)$, and the positive deviation of negative samples from the mean is also 1.46% $((198894-196026)/196026*100\% = 1.46\%)$. This indicates that the positive and negative samples have reached a balance relative to the mean, which shows that MPDA effectively balances the positive and negative samples.

In the training datasets of the 29 types of vulnerability augmented by MPDA, the number of samples augmented by the OSS component is shown in Table 6, the number of samples augmented by the GAN component is presented in Table 7, and the number of samples augmented by the FSS component is shown in Table 8.

As shown in Table 6, there are 91933 positive samples and 91621 negative samples among the samples augmented by the OSS component, with a total of 183554 samples. Among all MPDA-augmented samples, the proportions of the OSS-augmented samples are as follows. The proportion of positive samples is 47.6% $(91933/193157*100\% = 47.6\%)$, that of negative samples is 46.1% $(91621/198894*100\% = 46.1\%)$ and that of total samples is 46.8% $(183554/392051*100\% = 46.8\%)$.

As shown in Table 7, there are 22970 positive samples and 22892 negative samples among the samples augmented by the GAN component, with a total of 45862 samples. Among all MPDA-augmented samples, the proportions of the GAN-augmented samples are as follows: the proportion of positive samples is 11.9% $(22970/193157*100\% = 11.9\%)$, that of negative samples is 11.5% $(22892/198894*100\% = 11.5\%)$ and that of total samples is 11.7% $(45862/392051*100\% = 11.7\%)$.

As shown in Table 8, there are 87994 positive samples and 94052 negative samples among the samples augmented by the FSS component, with a total of 182046 samples. Among all MPDA-augmented samples, the proportions of the FSS-augmented samples are as follows: the proportion of positive samples is 45.6% $(87994/193157*100\% = 45.6\%)$, that of negative samples is 47.3% $(94052/198894*100\% = 47.3\%)$ and that of total samples is 46.4% $(182046/392051*100\% = 46.4\%)$.

Table 5 Augmentation result of the training set of the 29 types of common vulnerability in the raw dataset

Vulnerability type	Samples in raw training set	MPDA augmented training set		
		<i>P</i>	<i>N</i>	Total sample
CWE190	22256	37806	37806	75612
CWE191	17804	30242	30242	60484
CWE129	13656	23195	23195	46390
CWE89	11256	19800	19154	38954
CWE369	9401	15977	15977	31954
CWE789	5645	4563	4608	9171
CWE400	7404	8798	12550	21348
CWE113	6753	11479	11479	22958
CWE197	4327	3508	3508	7016
CWE134	3302	5589	5589	11178
CWE80	2321	2700	3413	6113
CWE643	700	829	1163	1992
CWE606	2251	3813	3813	7626
CWE36	1583	1899	2340	4239
CWE90	511	708	708	1416
CWE15	1583	2380	2333	4713
CWE23	1583	2295	2295	4590
CWE470	1583	1838	2301	4139
CWE78	1583	2400	2328	4728
CWE319	1553	2486	2619	5105
CWE81	1160	932	932	1864
CWE601	1160	1678	1678	3356
CWE83	1160	1728	1676	3404
CWE690	1600	2724	2724	5448
CWE476	948	1126	1608	2734
CWE563	747	1298	1293	2591
CWE259	422	618	618	1236
CWE398	412	483	679	1162
CWE506	207	265	265	530
Total	124871	193157	198894	392051

Note: The blue color indicates the augmented samples

4.2.2 Effect of MPDA in Advancing Vulnerability Detecting Models

To evaluate the effect of the proposed MPDA, we applied it to four deep learning models and the currently popular Transformer in vulnerability detection task, comparing to no data augmentation and three other individual data augmentation methods including oversample (here we used SMOTE), GAN and fuzzy sample. For the baseline, we did not perform any data augmentation operations on the training data, only the same preprocessing operations as our method. We totally applied three other individual data augmentation methods to the same vulnerability dataset to obtain different augmented datasets. Then we used distinct augmented datasets to train each of the four aforementioned deep learning models. Following training, vulnerability detection was performed on the same test datasets, and the

Table 6 Vulnerability samples Augmented by the OSS Component

Vulnerability type	OSS augmented samples		
	<i>P</i>	<i>N</i>	<i>Total</i>
CWE190	16803	16803	33606
CWE191	13441	13441	26882
CWE129	10309	10309	20618
CWE89	8800	8513	17313
CWE369	7101	7101	14202
CWE789	3651	3687	7338
CWE400	5578	5578	11156
CWE113	5102	5102	10204
CWE197	2807	2807	5614
CWE134	2484	2484	4968
CWE80	1517	1517	3034
CWE643	517	517	1034
CWE606	1695	1695	3390
CWE36	1085	1040	2125
CWE90	315	315	630
CWE15	1058	1037	2095
CWE23	1020	1020	2040
CWE470	1023	1023	2046
CWE78	1067	1035	2102
CWE319	1105	1164	2269
CWE81	746	746	1492
CWE601	746	746	1492
CWE83	768	745	1513
CWE690	1211	1211	2422
CWE476	715	715	1430
CWE563	577	575	1152
CWE259	275	275	550
CWE398	299	302	601
CWE506	118	118	236
Total	91933	91621	183554

average detection results of each method were calculated. Tables 9 and 10 show the specific results.

The following results can be observed from Tables 9 and 10. Our MPDA improves the detection accuracy on each of the four neural networks, with a minimum increase of 4.9 (95.5-90.6 = 4.9) in BiLSTM, while with a maximum increase of 10.5 (91.2-80.7 = 10.5) in LSTM, as shown in bold and underlined in column Acc. MPDA slightly decreases precision but significantly improves recall, resulting in an overall increase in the F1 score. This indicates that our method can maintain a certain level of accuracy while better capturing vulnerability samples and improving the comprehensiveness of vulnerability detection.

For the basic models LSTM and GRU, as shown in Table 9 in bold in column F1, our method greatly improves their detection performance, with an average increase of more than 5% ($(72.3-46.0) / 46.0 * 100\% = 57\%$) in the F1 score. For the more advanced mod-

Table 7 Vulnerability samples Augmented by the GAN Component

Vulnerability type	GAN augmented samples		
	<i>P</i>	<i>N</i>	<i>Total</i>
CWE190	4200	4200	8400
CWE191	3360	3360	6720
CWE129	2577	2577	5154
CWE89	2200	2128	4328
CWE369	1775	1775	3550
CWE789	912	921	1833
CWE400	1394	1394	2788
CWE113	1275	1275	2550
CWE197	701	701	1402
CWE134	621	621	1242
CWE80	379	379	758
CWE643	129	129	258
CWE606	423	423	846
CWE36	271	260	531
CWE90	78	78	156
CWE15	264	259	523
CWE23	255	255	510
CWE470	255	255	510
CWE78	266	258	524
CWE319	276	291	567
CWE81	186	186	372
CWE601	186	186	372
CWE83	192	186	378
CWE690	302	302	604
CWE476	178	178	356
CWE563	144	143	287
CWE259	68	68	136
CWE398	74	75	149
CWE506	29	29	58
Total	22970	22892	45862

els BiLSTM and BiGRU, as shown in Table 10 in bold and underlined in column F1, our method still increases their detection performance by at least 12% ($(93.1-82.9) / 82.9 * 100\% = 12\%$) based on already high detection results. We particularly noticed that for the BiGRU model, which achieved the best F1 score of 92.6 in the SySeVR method, MPDA can improve its performance of F1 score by 12.7% ($((93.2-82.7)/82.7*100\% = 12.7\%)$) in the vulnerability detection task. This demonstrates that our method has significant improvement effects on different types of neural networks and can continue to improve performance even on already excellent models.

Additionally, the evaluation results in using Transformer are presented in Table 11. From Table 11, it can be observed that compared to the other three data augmentation methods: Oversample (SMOTE), GAN, and Fuzzy sample, Our MPDA method improves the detection accuracy of Transformer to 92.9, with a maximum increase of 0.9 ($92.9-92.0 = 0.9$), as

Table 8 Vulnerability samples Augmented by the FSS Component

Vulnerability type	FSS augmented samples		
	<i>P</i>	<i>N</i>	<i>Total</i>
CWE190	16803	16803	33606
CWE191	16801	16801	33602
CWE129	10309	10309	20618
CWE89	11000	10641	21641
CWE369	7101	7101	14202
CWE789	912	921	1833
CWE400	1826	5578	7404
CWE113	6377	6377	12754
CWE197	701	701	1402
CWE134	3105	3105	6210
CWE80	804	1517	2321
CWE643	183	517	700
CWE606	1695	1695	3390
CWE36	543	1040	1583
CWE90	393	393	786
CWE15	1058	1037	2095
CWE23	1020	1020	2040
CWE470	560	1023	1583
CWE78	1067	1035	2102
CWE319	1105	1164	2269
CWE81	186	186	372
CWE601	932	932	1864
CWE83	960	931	1891
CWE690	1211	1211	2422
CWE476	233	715	948
CWE563	577	575	1152
CWE259	275	275	550
CWE398	110	302	412
CWE506	147	147	294
Total	87994	94052	182046

Table 9 Results on applying different data augmentation methods to improve LSTM & GRU

Methods	LSTM				GRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
Baseline	80.7	66.8	46.5	50.7	80.2	69.3	39.5	46.0
Oversample(SMOTE)	79.5	65.1	78.9	67.6	76.7	66.2	70.0	61.2
GAN	81.2	75.8	48.4	52.9	80.4	69.0	48.2	49.2
Fuzzy sample	80.2	63.9	51.7	53.3	78.0	65.8	42.3	46.4
Our method(MPDA)	91.2	80.9	96.7	87.4	85.7	69.1	79.9	72.3

Table 10 Results on applying different data augmentation methods to improve BiLSTM & BiGRU

Methods	BiLSTM				BiGRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
Baseline	<u>90.6</u>	89.2	81.2	<u>82.9</u>	90.1	87.0	82.6	82.7
Oversample(SMOTE)	91.4	80.0	97.3	86.4	90.7	79.9	95.4	85.1
GAN	92.1	88.0	85.9	84.3	90.7	88.9	80.1	81.6
Fuzzy sample	91.2	86.3	85.2	83.0	90.1	84.5	84.5	82.2
Our method(MPDA)	<u>95.5</u>	88.1	98.9	<u>93.1</u>	95.4	88.7	98.2	93.2

Table 11 Results on applying different data augmentation methods to improve Transformer

Methods	Transformer			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
Baseline	<u>92.0</u>	89.0	86.1	<u>84.9</u>
Oversample(SMOTE)	92.3	80.8	97.4	87.1
GAN	92.5	88.7	88.0	85.9
Fuzzy sample	92.0	87.6	87.1	85.0
Our method(MPDA)	<u>92.9</u>	82.3	98.0	<u>88.2</u>

shown in bold and underlined in column Acc. Furthermore, our MPDA method improves the Transformer detection F1 score to 88.2, with a maximum increase of 3.3 ($88.2 - 84.9 = 3.3$), as shown in bold and underlined in column F1. Thus, for the popular and excellent Transformer model, our MPDA method is still capable of improving its vulnerability detection performance, with the F1 score increased by 3.9% ($3.3/84.9 \times 100\% = 3.9\%$).

Our method shows great improvements in all metrics, in comparison with the other three individual data augmentation methods such as oversample, GAN, and fuzzy sample. This further confirms the effectiveness of our MPDA method and demonstrates its advantage in considering multiple data augmentation techniques comprehensively. Therefore, we can conclude that our data augmentation method is effective in advancing deep learning models for vulnerability detection and significantly improves the detection performance on various types of deep learning neural network.

4.2.3 Effect of OSS in MPDA

For the process of augmenting by oversampling in MPDA, we designed the OSS component, which could be considered as an optimized oversampling technology. In this section, we validate the effectiveness of OSS in data enhancement. This was demonstrated in two parts.

The first part aims to validate the effectiveness of our OSS technology. Initially, we applied the four different oversampling methods, such as SMOTE, Borderline-SMOTE, ADASYN, and our OSS, to augment the data, respectively, to obtain four different augmented datasets. Subsequently, we utilized the four augmented datasets to train, respectively, the four neural networks as detection models. Finally, we used the trained deep learning models to detect vulnerabilities on the same test dataset, to obtain and compare their detection results. Tables 12 and 13 show the specific results.

From Tables 12 and 13, it can be observed that the detection results achieved using four different oversampling methods across four types of neural network excel over the baseline

Table 12 Results on applying different oversampling methods to improve LSTM & GRU

Methods	LSTM				GRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
Baseline	80.7	66.8	46.5	50.7	80.2	69.3	39.5	46.0
SMOTE	79.5	65.1	78.9	67.6	76.7	66.2	70.0	61.2
Borderline_SMOTE	78.8	62.1	81.7	67.9	79.0	64.4	67.9	61.0
ADASYN	80.1	64.3	84.5	70.6	76.8	64.8	72.7	62.8
OSS	<u>84.7</u>	71.9	81.6	<u>73.7</u>	<u>81.4</u>	68.8	71.6	<u>65.8</u>

Table 13 Results on applying different oversampling methods to improve BiLSTM & BiGRU

Methods	BiLSTM				BiGRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
Baseline	90.6	89.2	81.2	82.9	90.1	87.0	82.6	82.7
SMOTE	91.4	80.0	97.3	86.4	90.7	79.9	95.4	85.1
Borderline_SMOTE	92.2	81.6	96.4	87.2	91.4	81.0	95.5	86.2
ADASYN	92.4	81.6	96.7	87.4	91.3	80.9	95.6	86.3
OSS	<u>93.6</u>	84.3	98.0	<u>89.6</u>	<u>92.8</u>	83.3	96.7	<u>85.5</u>

in terms of the F1 metric, as shown in bold in column F1. And our OSS achieves the best F1 values (73.7, 65.8, 89.6, 88.5) shown in bold and underlined on each neural network.

However, in both the LSTM and the GRU neural networks, the detection accuracy decreases when employing the other three oversampling technologies such as SMOTE, Borderline-SMOTE, and ADASYN, compared to the baseline, as shown by the accuracy values (79.5, 78.8, 80.1 compared with 80.7, 76.7, 79.0, 76.8 compared with 80.2) in bold in column Acc. in Table 12.

In contrast, adopting our OSS technology not only enhances the accuracy over the baseline and the other three oversampling technologies, as shown the accuracy values (84.7, 81.4, 93.6, 92.8) in bold and underlined in column Acc. in Tables 12 and 13, but also exceeds them in the F1 metric.

Our OSS leads to a minimum accuracy improvement of 1.2 ($81.4 - 80.2 = 1.2$) in GRU compared to baseline and a maximum accuracy improvement of 5.9 ($84.7 - 78.8 = 5.9$) in LSTM compared to Borderline-SMOTE, while a minimum F1 improvement of 2.2 ($88.5 - 86.3 = 2.2$ and $89.6 - 87.4 = 2.2$) in BiGRU and BiLSTM both compared with ADASYN, a maximum F1 improvement of 23 ($73.7 - 50.7 = 23$) on LSTM. Therefore, we can conclude that our OSS is efficient in the process of data augmenting by oversampling.

The second part aims to find out how advanced our OSS is in the data augmentation process by MPDA compared to the other traditional oversampling technologies. Firstly, we applied the four different data augmentation processes such as SMOTE + GAN + FSS, Borderline-SMOTE + GAN + FSS, ADASYN + GAN + FSS and our proposed MPDA (OSS+GAN+FSS) to augment the data, respectively, to obtain four different augmented datasets. Then, we used the four augmented datasets to train, respectively, the four neural networks as detection models. Finally, we used the trained deep learning models to detect vulnerabilities on the same test data set, obtain, and compare their detection results. Tables 14 and 15 show the specific results.

From Tables 14 and 15, it can be observed that our data augmentation process (OSS+GAN+FSS) emerges victorious compared to the other three. Due to the strategy

Table 14 Results on optimizing data augmentation with OSS to improve LSTM & GRU

Methods	LSTM				GRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
SMOTE+GAN+FSS	<u>85.6</u>	71.6	94.6	79.1	80.6	69.8	73.0	67.4
Borderline_SMOTE+GAN+FSS	86.7	74.0	89.0	<u>78.6</u>	80.5	70.9	79.2	<u>69.9</u>
ADASYN+GAN+FSS	85.7	71.0	94.6	79.4	81.1	67.8	77.9	68.5
OSS+GAN+FSS(MPDA)	<u>91.2</u>	80.9	96.7	<u>87.4</u>	85.7	69.1	79.9	<u>72.3</u>

Table 15 Results on optimizing data augmentation with OSS to improve BiLSTM & BiGRU

Methods	BiLSTM				BiGRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
SMOTE+GAN+FSS	92.9	84.3	95.3	88.5	92.2	82.8	95.5	87.7
Borderline_SMOTE+GAN+FSS	92.9	82.2	97.7	88.5	93.1	82.3	98.4	88.8
ADASYN+GAN+FSS	93.1	83.3	97.3	89.5	<u>93.2</u>	83.0	97.9	89.6
OSS+GAN+FSS(MPDA)	95.5	88.1	98.9	93.1	<u>95.4</u>	88.7	98.2	93.2

adjustments in OSS, our OSS+GAN+FSS method surpasses the results obtained by using any of the other three oversampling technologies individually, such as SMOTE+GAN+FSS, Borderline-SMOTE+GAN+FSS, and ADASYN+GAN+FSS, both in accuracy and in F1 score, denoted by values in bold in column Acc. and F1 in the two tables. The results showed that, compared to ADASYN + GAN + FSS, the improvement in the accuracy rate of the BiGRU model achieved by using our method with OSS is the smallest, which is 2.2 ($95.4 - 93.2 = 2.2$). However, compared to SMOTE+GAN+FSS, the improvement in the accuracy rate of the LSTM model is the largest, which is 5.6 ($91.2 - 85.6 = 5.6$). Meanwhile, compared to Borderline-SMOTE+GAN+FSS, when using our method with OSS, the improvement in the F1 performance of the GRU is the smallest, which was 2.4 ($73.2 - 39.9 = 2.4$). However, the improvement in LSTM F1 performance is the largest, which is 8.8 ($87.3 - 78.6 = 8.8$), denoted by values in bold and underlined.

This shows that our OSS is very important in the data augmentation process. Our OSS performs the best compared to oversampling methods and significantly improves the performance of the detection model. By selecting different strategies for different types of vulnerabilities and neural networks, it can effectively optimize the data augmentation method under different scenarios, resulting in higher-quality training data, improving the accuracy and robustness of deep learning models for vulnerability detection. This underscores the critical role of strategic selection in our methodology, ensuring the enhanced versatility and efficacy of our MPDA method for vulnerability detection tasks.

4.2.4 Effect of FSS in MPDA

Is the FSS component effective and advanced? We validate the effect of FSS mainly in two parts. The first part aims to validate the effect of our FSS. The second part aims to find out how advanced our FSS is compared to traditional fuzzy sampling technology.

In the first part, our focus is on validating the effectiveness of our FSS component in the data augmentation process. So we did the experiment with different data augmentation approaches such as the baseline (not use any data augmentation technology as mentioned above), both the traditional oversample and GAN(SMOTE+GAN), MPDA without

Table 16 Results on applying FSS to improve LSTM & GRU

Methods	LSTM				GRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
Baseline	80.7	66.8	46.5	50.7	80.2	69.3	39.5	46.0
SMOTE+ GAN	80.0	65.7	79.2	66.7	76.4	69.0	73.3	63.8
OSS+GAN	82.4	68.9	83.3	71.6	78.1	69.9	76.5	67.1
OSS+GAN+FSS(MPDA)	<u>91.2</u>	80.9	96.7	<u>87.4</u>	<u>85.7</u>	69.1	79.9	<u>72.3</u>

Table 17 Results on applying FSS to improve BiLSTM & BiGRU

Methods	BiLSTM				BiGRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
Baseline	90.6	89.2	81.2	82.9	90.1	87.0	82.6	82.7
SMOTE+GAN	91.4	79.7	97.6	86.5	90.8	80.7	94.2	84.9
OSS+GAN	92.8	83.9	94.5	87.1	92.7	83.0	96.2	87.9
OSS+GAN+FSS(MPDA)	<u>95.5</u>	88.1	98.9	<u>93.1</u>	<u>95.4</u>	88.7	98.2	<u>93.2</u>

FSS(OSS+GAN), and MPDA(OSS+GAN+FSS), respectively. First, we applied the four aforementioned data augmentation processes separately to augment the data, resulting in four distinct training datasets. Subsequently, we individually trained four neural networks as detection models with these four datasets. Ultimately, we used trained deep learning models to detect vulnerabilities within the same test set. Specific results are shown in Tables 16 and 17.

From Tables 16 and 17, it can be observed that the detection results of our MPDA using FSS in four types of neural networks excel over the baseline and the other two data augmentation processes not using FSS in both accuracy and F1. The values are shown in bold in columns Acc. and F1 in the two tables.

Additionally, in each neural network, our MPDA with FSS achieves the best accuracy values(91.2, 85.7, 95.5, 95.4) shown in bold and underlined in column Acc. in the two tables and also achieves the best F1 values(87.4, 72.3, 93.1, 93.2) shown in underline and bold in column F1 in the two tables.

Compared to the combination of OSS+GAN (that is, MPDA without FSS), the combination of OSS+GAN+FSS (that is, MPDA) improves the accuracy of the BiLSTM (or BiGRU) vulnerability detection performance by a minimum margin of 2.7 ($95.5-92.8 = 2.7$ or and $95.4-92.7 = 2.7$). In contrast, compared to the combination of SMOTE+GAN, OSS+GAN+FSS (that is, MPDA) improves the accuracy of the vulnerability detection performance of LSTM by a maximum margin of 11.2 ($91.2-80.0 = 11.2$). Furthermore, compared to OSS + GAN (that is, MPDA without FSS), OSS+GAN+FSS (namely MPDA) improves the F1 score of the vulnerability detection performance of GRU by a minimum of 5.2 ($72.3-67.1 = 5.2$). In contrast, compared to the combination of SMOTE+GAN, OSS+GAN+FSS (namely MPDA) increases the F1 score for the vulnerability detection performance of LSTM to a maximum of 20.7 ($87.4-66.7 = 20.7$).

The second part aims to find out how advanced our FSS is in the data augmentation process compared to other traditional data augmentation technologies. Firstly, we, respectively, applied the four different data augmentation processes such as SMOTE+GAN+fuzzy sample, SMOTE+GAN+FSS, OSS+GAN+fuzzy sample and our proposed MPDA (OSS+GAN+FSS) to augment the data, to obtain four different augmented datasets. Then,

Table 18 Results on optimizing data augmentation with FSS to improve LSTM & GRU

Methods	LSTM				GRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
SMOTE+GAN+Fuzzy sample	82.3	69.6	86.5	73.3	78.5	63.7	67.5	61.1
SMOTE+GAN+FSS	85.6	71.6	94.6	79.1	80.6	69.8	73.0	67.4
OSS+GAN+Fuzzy sample	84.3	70.8	87.7	76.1	83.0	70.0	67.2	64.0
OSS+GAN+FSS(MPDA)	91.2	80.9	96.7	87.4	85.7	69.1	79.9	72.3

Table 19 Results on optimizing data augmentation with FSS to improve BiLSTM & BiGRU

Methods	BiLSTM				BiGRU			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
SMOTE+GAN+fuzzy sample	91.8	81.9	94.5	86.3	92.2	83.1	95.1	86.2
SMOTE+GAN+FSS	92.9	84.3	95.3	88.5	92.2	82.8	95.5	87.7
OSS+GAN+Fuzzy sample	92.3	82.7	95.7	86.8	93.5	84.5	96.0	88.8
OSS+GAN+FSS(MPDA)	95.5	88.1	98.9	93.1	95.4	88.7	98.2	93.2

we used the four augmented datasets to respectively train the four neural networks (LSTM, GRU, BiLSTM and BiGRU) as detection models. Finally, we used the trained deep learning models to detect vulnerabilities on the same test dataset, to obtain and compare their detection results. The specific results are presented in Tables 18 and 19.

From Tables 18 and 19, it can be observed that our data augmentation process (OSS+GAN+FSS) emerges victorious compared to the other three. Because of the strategy adjustments in FSS, our OSS+GAN+FSS method surpasses the results obtained by using any of the other three data augmentation processes individually, such as SMOTE+GAN+fuzzy sample, SMOTE+GAN+FSS and OSS+GAN+fuzzy sample, in both accuracy and F1, as shown the values in bold in column Acc. and F1 in the two tables.

The values in bold and underlined in Tables 18 and 19 show that compared to the combination of OSS+GAN+fuzzy sample, the combination of OSS+GAN+FSS (that is, MPDA) improves the accuracy of BiGRU vulnerability detection performance by a minimum margin of 1.9 ($95.4 - 93.5 = 1.9$). In contrast, compared to the combination of SMOTE+GAN+fuzzy sample, OSS+GAN+FSS (that is, MPDA) improves the accuracy of the vulnerability detection performance of LSTM by a maximum margin of 8.9 ($91.2 - 82.3 = 8.9$). Furthermore, compared to the OSS+GAN+fuzzy sample approach, OSS+GAN+FSS (namely MPDA) improves the F1 score of the vulnerability detection performance of BiGRU by a minimum of 4.4 ($93.2 - 88.8 = 4.4$). In contrast, compared to the SMOTE+GAN+fuzzy sample method, OSS+GAN+FSS (namely MPDA) increases the F1 score of vulnerability detection performance of LSTM to a maximum of 14.1 ($87.4 - 73.3 = 14.1$).

This shows that our FSS is also very important in the data augmentation process. It performs better in comparison with others and significantly improves the performance of the vulnerability detection model. It can optimize the data augmentation method under different scenarios, resulting in higher quality training data and improving the accuracy and robustness of the vulnerability detection model. This highlights the importance of strategy selection in the process of data augmentation by fuzzy sample, making our data augmentation method more versatile and effective in the task of vulnerability detection.

4.2.5 Applying MPDA to Improve Deep Learning Model for Software Vulnerability Detection Compared with Achilles

How advanced is our proposed MPDA approach in software vulnerability detection? In this section, we evaluated the effectiveness of MPDA in improving deep learning-based vulnerability detection methods compared to Achilles.

In the detection task, we used BiLSTM as the deep learning model, used both raw and MPDA enhanced data for model training. Subsequently, we compared the predictions of these two models with the results obtained by Achilles. The comparative detection results of accuracy for 29 types of vulnerabilities are shown in Fig. 2.

As shown in Fig. 2, when training the BiLSTM model using original data without data augmentation, the detection performance is weaker than Achilles in 22 of the 29 types of vulnerabilities. However, when training the BiLSTM model using MPDA-augmented data, the detection performance surpasses Achilles in 27 out of the 29 types of vulnerabilities. This clearly indicates that the MPDA-enhanced training dataset significantly improves the model performance in vulnerability detection and results in noticeable improvements in overall detection capabilities. Furthermore, the three methods all achieve an accuracy of 1 in detecting the CWE398 vulnerabilities. This indicates that these methods perform equally well for this category. However, in case of CWE259 vulnerabilities, our method slightly lags behind Achilles, although it shows improvement in the detection results. This suggests that further adjustments and optimizations should be made to our method in future work to enhance its detection capabilities for this type of vulnerability.

In conclusion, the enhanced training dataset through MPDA significantly improves the model's performance in detection and outperforms Achilles in most vulnerabilities. This demonstrates the effect of our data augmentation method. However, further improvements and optimizations should be explored in detecting certain specific types of vulnerability. The results highlight the potential of our approach and underscore the importance of ongoing research to achieve even better performance in vulnerability detection.

4.2.6 Effect of MPDA on Small-sample Vulnerability Dataset

Is the MPDA method effective for a small number of vulnerability data samples?

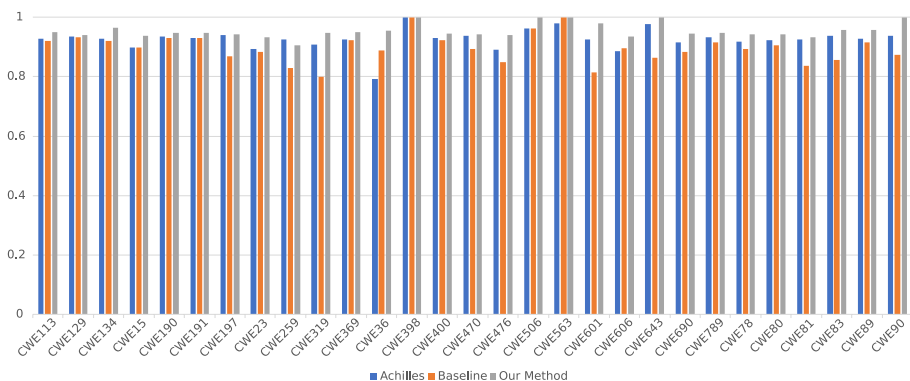


Fig. 2 Results of accuracy on applying MPDA to improve deep learning based vulnerability detection method compared with Achilles

In addition to the 29 types of vulnerability commonly used, the NIST Juliet dataset also includes other types of vulnerability data, all of which contain limited source code samples, have not been researched, and are too few to support effective deep learning detection. We consider that the accuracy of the detection result that is greater than 0.8 is effective and applicable. This is because: the research by Li et al. (2018) pointed out that under a balanced dataset (for example, when the proportion of vulnerability samples is approximately 30%), a model with an accuracy rate greater than or equal to 0.8 can significantly reduce the false positive rate ($FP < 0.15$) and reduce the manual verification workload by more than 60%, thus saving the cost of manual review. This research achieved an accuracy rate of 82.3% on real datasets (NVD and SARD) and is considered an effective threshold in the industry. In NIST Special Publication 800-53, it is recommended that the accuracy rate of automated vulnerability detection tools be greater than or equal to 0.7.

In empirical experiments, we found that in the NIST Juliet dataset, for each of the 29 commonly used types of vulnerability data, the method samples of the source code are more than 250, and the accuracy of detection result is greater than 0.8, but for each of the other types, the method samples of source code are less than 250, and the accuracy of detection result is less than 0.8. So we defined the upper bound of few samples of vulnerability data in the method-level source code as 250, and according to this, we chose a subset of the NIST Juliet dataset as the small-sample dataset which contains 51 types of vulnerability data, to validate the effect of the MPDA method on few samples of vulnerability data.

With the above small-sample vulnerability dataset, we trained LSTM as the downstream vulnerability detection model. The training set was augmented with MPDA, resulting in a new training set. So we get two training sets. One is the original training set from the small-sample vulnerability dataset. The other is the MPDA-augmented training set. We trained the downstream LSTM model with the two training sets to perform vulnerability detection on the same testing set, respectively. Based on the average results of more than five experiments, we found that 18 of 51 types of vulnerability enhanced by MPDA improved the average accuracy of the detection model to achieve more than 0.8. The average detection results of these 18 types of vulnerabilities are presented in Table 20. In addition, Fig. 3 illustrates the comparative detection results for 5 of 18 types of vulnerabilities augmented by MPDA with an average improvement in the accuracy of the detection model greater than 0.1.

As shown in Table 20, compared to models trained using non-augmented data, models trained with data augmented by MPDA have shown a significant improvement in accuracy, precision, recall and F1 score performance. And, as shown in Fig. 3, on vulnerabilities CWE325, CWE478, CWE511, CWE605 and CWE759, the MPDA approach improves the accuracy of the detection model greater than 0.1. It suggests that MPDA is effective for some types of vulnerability in a small sample dataset, but there is also space to improve MPDA to be effective in other types of vulnerability.

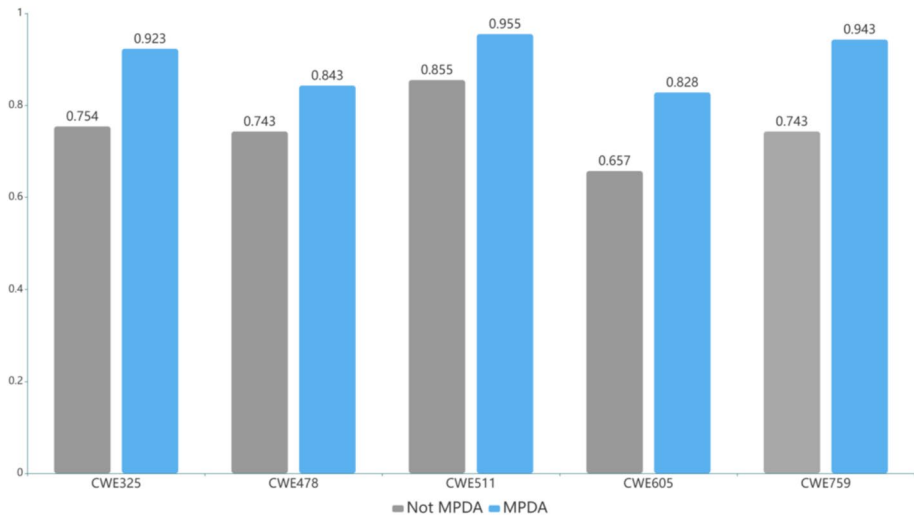
4.3 More Discussions

4.3.1 Computational Cost Comparison among Distinct Code Representation Methods

During the data preprocessing phase of MPDA, with the aim of facilitating the selection of a suitable code representation method, the computational costs of data representation for such representation methods as Word2Vec, CodeBERT, UniXcoder and CoderT5+ were

Table 20 Detection results of LSTM on small-sample vulnerability dataset enhanced by MPDA

Method	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
Not MPDA	89.3	66.4	64.7	64.0
MPDA	94.4	82.1	82.9	81.1

**Fig. 3** Comparative results on 5 types of vulnerabilities with an accuracy improvement of the detection model greater than 0.1

compared on the dataset of 29 common vulnerabilities within the NIST Juliet 1.2 Java test suite. For the sake of a fair experiment and to avoid random results, we performed the data representation experiment for each method ten times and obtained the mean of the ten results. Moreover, in the case of Word2Vec, we also evaluated its computational cost when the length of its token vector was the same as that of CodeBERT, UniXcoder and CodeT5+ (that is, 768 dimensions).

The experimental results are presented in Table 21. It can be seen that when preprocessing the same dataset and performing the MPDA data representation process, Word2Vec requires the lowest computing power. It does not need a GPU and can run with just two cores of the 6226R CPU. When the token vector is 150 dimensional, it takes the shortest time to complete the data representation, which is 7 minutes and 14 seconds. Even when the token vector is increased to 768 dimensions, which is consistent with other models, the time taken is only 11 minutes and 32 seconds, which is nearly half that of other models. In contrast, CodeBERT, Unixcoder, and CodeT5+ demand high computing power and cannot operate properly without a Graphics Processing Unit (GPU) and also consume more time. Among them, CodeT5+ takes the longest time, reaching 28 minutes and 23 seconds. Therefore, we can conclude that compared to CodeBERT, Unixcoder and CodeT5+, Word2Vec incurs the lowest computational cost for the same data representation task.

4.3.2 Effect of MPDA on Advancing Vulnerability Detection Models under the Same Data Scale

To evaluate the effect of MPDA on improving the performance of vulnerability detection models by improving data quality rather than simply increasing the amount of data, we conducted an experiment to evaluate the performance improvement of vulnerability detection models by MPDA in training sets of the same scale. We randomly extracted four training sets of sizes N and M (where $N < M$) from the raw vulnerability dataset and the augmented vulnerability data, namely N_{raw} , N_{aug} , M_{raw} , and M_{aug} , as shown in Table 22. The training sets N_{raw} and N_{aug} totally contain 41,611 original vulnerability samples and augmented vulnerability samples, respectively. Meanwhile, the training sets M_{raw} and M_{aug} totally contain 124,871 original vulnerability samples and augmented vulnerability samples, respectively.

To evaluate the effectiveness of MPDA in enhancing the vulnerability detection performance of neural network models on training sets of the same scale N , we trained each neural network model using two training sets, N_{raw} and N_{aug} , both of scale N . The detection performance of each model after training on N_{raw} and N_{aug} is shown in Table 23. In addition, to evaluate the effectiveness of MPDA in improving the vulnerability detection performance of neural network models on training sets of a larger but identical scale M , we trained each neural network model using two training sets, M_{raw} and M_{aug} , both of scale M . The detection performance of each model after training on M_{raw} and M_{aug} is presented in Table 24.

For LSTM, when trained with the datasets N_{raw} and N_{aug} of the same scale N (41611), LSTM trained on the enhanced dataset N_{aug} demonstrated better performance in terms of recall and F1 compared to LSTM trained on the raw dataset N_{raw} . As shown in the blue bold results in Table 23, the recall increased by 67.8% ($(66.8 - 39.8) / 39.8 * 100\% = 67.8\%$), and F1 improved by 25.9% ($(55.0 - 43.7) / 43.7 * 100\% = 25.9\%$). When trained with the datasets M_{raw} and M_{aug} of the same scale M (124871), the LSTM trained on the augmented dataset M_{aug} witnessed a 73.9% ($(80.7 - 46.4) / 46.4 * 100\% = 73.9\%$) increase in recall, and a 31.4% ($(66.1 - 50.3) / 50.3 * 100\% = 31.4\%$) enhancement in F1 compared to the LSTM trained on the raw dataset M_{raw} . This indicates that MPDA effectively enhances the quality of the data. Under the condition of the same data volume, MPDA can still effectively improve the vulnerability detection performance of the LSTM model, especially in terms of recall and F1. Furthermore, as the scale of the dataset increases (for example, from $N = 41611$ to $M = 124871$), this improvement effect gradually strengthens (for example, the improvement in F1 increases from 25.9% to 31.4%, and the improvement in recall increases from 67.8% to 73.9%).

Table 21 Computational costs of data representation for different representation methods

Representation method	Time cost	Computing power used
Word2Vec(150D token vector)	0:07:14	2 cores of 6226R CPU
Word2Vec(768D token vector)	0:11:32	2 cores of 6226R CPU
CodeBERT(768D token vector)	0:22:55	2 cores of 6226R CPU + RTX 3090 GPU *2
UniXcoder(768D token vector)	0:24:33	2 cores of 6226R CPU + RTX 3090 GPU *2
CodeT5+(768D token vector)	0:28:23	2 cores of 6226R CPU + RTX 3090 GPU *2

Table 22 Training sets of vulnerability dataset with N and M samples before and after MPDA augmentation ($N < M$)

Vulnerability name	Training set with N samples		Training set with M samples	
	N_{raw}	N_{aug}	M_{raw}	M_{aug}
CWE190	7418	7418	22256	22256
CWE191	5934	5934	17804	17804
CWE129	4552	4552	13656	13656
CWE89	3752	3752	11256	11256
CWE369	3133	3133	9401	9401
CWE789	2468	2468	7404	7404
CWE400	2251	2251	6753	6753
CWE113	1881	1881	5645	5645
CWE197	1442	1442	4327	4327
CWE134	1100	1100	3302	3302
CWE80	773	773	2321	2321
CWE606	750	750	2251	2251
CWE643	533	533	1600	1600
CWE36	527	527	1583	1583
CWE15	527	527	1583	1583
CWE23	527	527	1583	1583
CWE470	527	527	1583	1583
CWE78	527	527	1583	1583
CWE90	517	517	1553	1553
CWE319	386	386	1160	1160
CWE81	386	386	1160	1160
CWE601	386	386	1160	1160
CWE83	316	316	948	948
CWE690	249	249	747	747
CWE476	233	233	700	700
CWE563	170	170	511	511
CWE259	140	140	422	422
CWE398	137	137	412	412
CWE506	69	69	207	207
Total	41611	41611	124871	124871

Note: N_{raw} represents N raw samples, N_{aug} represents N augmented samples, M_{raw} represents M raw samples, and M_{aug} represents M augmented samples

For GRU, when trained with the datasets N_{raw} and N_{aug} of the same scale N (41611), GRU trained on the enhanced dataset N_{aug} showed better performance in terms of recall and F1 compared to GRU trained on the raw dataset N_{raw} . As shown in the blue bold results in Table 23, the recall increased by 80.7% ($(51.5 - 28.5) / 28.5 * 100\% = 80.7\%$), and F1 improved by 33.5% ($(47.4 - 35.5) / 35.5 * 100\% = 33.5\%$). When trained with the datasets M_{raw} and M_{aug} of the same scale M (124871), the GRU trained on the augmented dataset M_{aug} witnessed a 58.2% ($(61.4 - 38.8) / 38.8 * 100\% = 58.2\%$) increase in recall, and a 22.5% ($(55.0 - 44.9) / 44.9 * 100\% = 22.5\%$) enhancement in F1 compared to the GRU trained on the raw dataset M_{raw} . This indicates that MPDA effectively enhances the qual-

Table 23 Impact of MPDA on detection performance of various models trained with N samples

Models	N_{raw}				N_{aug}			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
LSTM	77.8	62.1	39.8	43.7	71.7	55.7	66.8	55.0
GRU	76.7	64.6	28.5	35.5	72.4	59.5	51.5	47.4
BiGRU	86.6	82.3	71.8	72.6	84.4	72.7	84.6	74.6
BiLSTM	88.4	84.2	79.7	78.6	87.4	76.4	90.3	80.2
Transformer	90.1	84.8	84.0	81.1	88.7	77.7	91.3	80.9
BiGRUatt	87.0	78.8	73.8	72.6	86.1	73.4	87.7	76.9

Table 24 Impact of MPDA on detection performance of various models trained with M samples

Models	M_{raw}				M_{aug}			
	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>	<i>Acc.</i>	<i>Pre.</i>	<i>Rec.</i>	<i>F1</i>
LSTM	80.8	67.7	46.4	50.3	76.8	62.2	80.7	66.1
GRU	79.8	68.4	38.8	44.9	75.6	62.0	61.4	55.0
BiGRU	90.7	87.2	83.3	82.9	89.9	79.5	92.6	83.6
BiLSTM	91.2	87.0	85.1	83.6	91.1	80.3	94.7	85.2
Transformer	92.3	88.6	87.3	85.3	91.8	81.1	96.3	86.3
BiGRUatt	91.1	87.2	84.0	82.8	91.4	80.5	96.1	85.8

ity of the data. Under the condition of the same data volume, MPDA can still effectively improve the vulnerability detection performance of the GRU model, especially in terms of recall and F1. However, as the scale of the dataset increases (for example, from $N = 41611$ to $M = 124871$), this improvement effect gradually weakens (for example, the improvement in F1 decreases from 33.5% to 22.5%, and the improvement in recall decreases from 80.7% to 58.2%).

For BiGRU, when trained with the datasets N_{raw} and N_{aug} of the same scale N (41611), BiGRU trained on the enhanced dataset N_{aug} showed better performance in terms of recall and F1 compared to BiGRU trained on the raw dataset N_{raw} . As shown in the blue bold results in Table 23, the recall increased by 17.8% ($(84.6 - 71.8) / 71.8 * 100\% = 17.8\%$), and F1 improved by 2.8% ($(74.6 - 72.6) / 72.6 * 100\% = 2.8\%$). When trained with the datasets M_{raw} and M_{aug} of the same scale M (124871), the BiGRU trained on the augmented dataset M_{aug} witnessed an 11.2% ($(92.6 - 83.3) / 83.3 * 100\% = 11.2\%$) increase in recall, and a 0.8% ($(83.6 - 82.9) / 82.9 * 100\% = 0.8\%$) enhancement in F1 compared to the BiGRU trained on the raw dataset M_{raw} . This indicates that MPDA effectively enhances the quality of the data. Under the condition of the same data volume, MPDA can still effectively improve the vulnerability detection performance of the BiGRU model, especially in terms of recall and F1. However, as the scale of the dataset increases (for example, from $N = 41611$ to $M = 124871$), this improvement effect gradually weakens (for example, the improvement in F1 decreases from 2.8% to 0.8%, and the improvement in recall decreases from 17.8% to 11.2%).

For BiLSTM, when trained with the datasets N_{raw} and N_{aug} of the same scale N (41611), BiLSTM trained on the enhanced dataset N_{aug} showed better performance in terms of recall and F1 compared to BiLSTM trained on the raw dataset N_{raw} . As shown in the blue bold

results in Table 23, the recall increased by 13.3% ($(90.3 - 79.7) / 79.7 * 100\% = 13.3\%$), and F1 improved by 2.0% ($(80.2 - 78.6) / 78.6 * 100\% = 2.0\%$). When trained with the datasets M_{raw} and M_{aug} of the same scale M (124871), the BiLSTM trained on the augmented dataset M_{aug} witnessed an 11.3% ($(94.7 - 85.1) / 85.1 * 100\% = 11.3\%$) increase in recall, and a 1.9% ($(85.2 - 83.6) / 83.6 * 100\% = 1.9\%$) enhancement in F1 compared to the BiLSTM trained on the raw dataset M_{raw} . This indicates that MPDA effectively enhances the quality of the data. Under the condition of the same data volume, MPDA can still effectively improve the vulnerability detection performance of the BiLSTM model, especially in terms of recall and F1. However, as the scale of the dataset increases (for example, from $N = 41611$ to $M = 124871$), this improvement effect gradually weakens (for example, the improvement in F1 decreases from 2.0% to 1.9%, and the improvement in recall decreases from 13.3% to 11.3%).

For Transformer, when trained with the datasets N_{raw} and N_{aug} of the same scale N (41611), Transformer trained on the enhanced dataset N_{aug} showed better performance in terms of recall and F1 compared to Transformer trained on the raw dataset N_{raw} . As shown in the blue bold results in Table 23, the recall increased by 8.7% ($(91.3 - 84.0) / 84.0 * 100\% = 8.7\%$), and F1 improved by -0.2% ($(80.9 - 81.1) / 81.1 * 100\% = -0.2\%$). When trained with the datasets M_{raw} and M_{aug} of the same scale M (124871), the Transformer trained on the augmented dataset M_{aug} witnessed a 10.3% ($(96.3 - 87.3) / 87.3 * 100\% = 10.3\%$) increase in recall and a 1.2% ($(86.3 - 85.3) / 85.3 * 100\% = 1.2\%$) enhancement in F1 compared to the Transformer trained on the raw dataset M_{raw} . This indicates that MPDA effectively enhances the quality of the data. Under the condition of the same data volume, MPDA can still effectively improve the vulnerability detection performance of the Transformer model, especially in terms of recall. Furthermore, as the scale of the dataset increases (for example, from $N = 41611$ to $M = 124871$), this improvement effect gradually strengthens (for example, the improvement in F1 increases from -0.2% to 1.2%, and the improvement in recall increases from 8.7% to 10.3%).

For BiGRUatt, which is an enhanced version of BiGRU with an incorporated attention mechanism, when trained with the datasets N_{raw} and N_{aug} of the same scale N (41611), BiGRUatt trained on the enhanced dataset N_{aug} demonstrated better performance in terms of recall and F1 compared to BiGRUatt trained on the raw dataset N_{raw} . As shown in the blue bold results in Table 23, the recall increased by 18.8% ($(87.7 - 73.8) / 73.8 * 100\% = 18.8\%$), and F1 improved by 5.9% ($(76.9 - 72.6) / 72.6 * 100\% = 5.9\%$). When trained with the datasets M_{raw} and M_{aug} of the same scale M (124871), the BiGRUatt trained on the augmented dataset M_{aug} witnessed a slight improvement of 0.3% in accuracy ($(91.4 - 91.1) / 91.1 * 100\% = 0.3\%$), a 14.4% ($(96.1 - 84.0) / 84.0 * 100\% = 14.4\%$) increase in recall, and a 3.6% ($(85.8 - 82.8) / 82.8 * 100\% = 3.6\%$) enhancement in F1 compared to the BiGRUatt trained on the raw dataset M_{raw} . This indicates that MPDA effectively enhances the quality of the data. Under the condition of the same data volume, MPDA can still effectively improve the vulnerability detection performance of the model, especially in terms of recall and F1. However, as the scale of the dataset increases (for example, from $N = 41611$ to $M = 124871$), this improvement effect gradually weakens (for example, the improvement in F1 decreases from 5.9% to 3.6%, and the improvement in recall drops from 18.8% to 14.4%).

We found that, under the same data scale, for simple models such as LSTM and GRU, MPDA significantly improves their recall and F1 scores in vulnerability detection. Furthermore, as the size of the dataset increases significantly, the improvement in LSTM recall and

F1 scores by MPDA gradually increases, while the improvement in those of GRU gradually decreases. For more complex models such as BiLSTM, BiGRU, BiGRUatt, and Transformer, the improvement in the recall and F1 scores of vulnerability detection by MPDA is less significant compared to that of simple models. However, as the size of the dataset increases significantly, the improvement in the recall and F1 scores of BiLSTM, BiGRU, and BiGRUatt by MPDA gradually decreases, while the improvement in those of Transformer gradually increases. On a smaller training set of size N , the improvement in the F1 score of Transformer by MPDA is still negative. However, on a larger training set of size M , the improvement in the F1 score of the Transformer has become positive.

It is worth noting that the models improved by MPDA mentioned above can be further optimized through hyperparameter tuning in the future to maximize their performance.

4.3.3 Statistical Significance of Experimental Results

During the experiment, to minimize the influence of randomness, each experiment was repeated 10 times. We conducted a statistical analysis on the experimental results of MPDA's improvement in the performance of these five models: LSTM, GRU, BiLSTM, BiGRU and Transformer, calculating the T-statistic, the P-value, and the effect size (cohen_d), as shown in Table 25.

In Table 25, except for the precision of LSTM and GRU as well as the accuracy of GRU, the absolute values of the T-statistic for the four performance metrics of the five models are greater than 2, indicating statistical significance. In addition, the T-statistic values of recall and F1 for the five models are mainly much greater than 2, clearly demonstrating statistical significance. The P-values of the four performance metrics for all five models are all less than 0.05, and the P-values of recall and F1 for the five models are all far less than 0.05, with some even approaching 0, which illustrates significant differences and a high credibility that MPDA can improve the recall and F1 performance of each model. Furthermore, the effect sizes of the Recall for the five models and F1 for LSTM are all greater than 0.8, indicating a large effect and demonstrating the significant improvement effect of MPDA on these performance metrics of these models. The effect sizes of F1 for GRU, BiLSTM, BiGRU, and Transformer are greater than 0.5 and smaller than 0.8, indicating a medium effect and reflecting the moderate improvement effect of MPDA on these performance metrics of these models. The effect sizes of the accuracy for LSTM, BiLSTM, BiGRU, and Transformer fall within the range of a small effect, suggesting that MPDA has a relatively small improvement in the accuracy performance of these models.

5 Threats to Validity

Firstly, the current selection strategy can only provide the optimal solution for 29 types of commonly used vulnerabilities and 18 of 51 types of small-sample vulnerabilities in the Java language. For other unknown vulnerabilities, since we have not conducted experiments on them, we can only offer suboptimal solutions. This limitation may result in the detection performance not reaching its maximum potential for these unknown vulnerabilities.

Secondly, we need to recognize that the Java language may not fully represent the characteristics and types of vulnerabilities in other programming languages. Therefore, when

Table 25 Statistical significance of MPDA-advanced deep learning models' performance results

Model	Metric	T-statistic	P-value	Effect size
LSTM	Accuracy	3.474046	0.001686	<i>0.454796</i>
	Precision	0.932987	0.358803	0.1583
	Recall	9.635894	0	<u>2.045513</u>
	F1	7.062345	0	<u>1.179862</u>
GRU	Accuracy	-0.364883	0.71794	-0.058467
	Precision	-1.056529	0.299758	-0.089603
	Recall	8.696535	0	<u>1.404223</u>
	F1	6.511638	0	<u>0.74274</u>
BiLSTM	Accuracy	3.132896	0.004033	<i>0.46095</i>
	Precision	-3.933298	0.000502	-0.696156
	Recall	9.296161	0	<u>1.831081</u>
	F1	4.862609	0.00004	<u>0.640839</u>
BiGRU	Accuracy	3.08952	0.004496	<i>0.409226</i>
	Precision	-4.91743	0.000035	-0.835166
	Recall	10.982969	0	<u>1.99788</u>
	F1	5.270596	0.000013	<u>0.670993</u>
Transformer	Accuracy	1.27674	0.212182	<i>0.267667</i>
	Precision	-4.269125	0.000203	-0.936784
	Recall	9.915217	0	<u>2.37282</u>
	F1	2.558462	0.01621	<u>0.541217</u>

Note: An absolute value of the T-statistic greater than 2 indicates a relatively large difference and may possess statistical significance, and the corresponding T-values are shown in bold. A P-value less than 0.05 indicates a significant difference. When P-value displays 0, it means that the valid value cannot be displayed within 6 decimal places. An effect size d that is greater than or equal to 0.2 and less than 0.5 indicates a small effect, and the corresponding d -values are shown in italics. When d is greater than or equal to 0.5 and less than 0.8, it indicates a medium effect, and the corresponding d -values are shown in bold. If d is greater than or equal to 0.8, it indicates a large effect, and the corresponding d -values are shown in bold and underlined

applying our method to other programming languages, careful consideration of the specific features and data distributions of different languages is essential, and adjustments and optimizations should be made accordingly.

Finally, for the sake of overall coherence and efficiency, we made certain trade-offs in the strategy selection process, which might lead to overlooking specific vulnerabilities and not achieving the optimal solution for them. These trade-offs were made to strike a balance between performance and resource constraints, and as a result, some vulnerabilities might not receive the full benefit of our approach.

6 Related Work

Many significant researches have been done on software vulnerability detection (Shiri Harzevili et al. 2024; Liang et al. 2025). As early as 2003, NIST released the CWEs to provide a standardized classification of software vulnerabilities. In 2023, MITRE published the 25 most dangerous in CWEs as evaluation standards for vulnerability detection tools. These standards and evaluation tools provide a foundational framework and guidance for vulner-

ability detection. In addition, many static vulnerability scanners have emerged and gained popularity in industry, such as Programming Mistake Detector (PMD), FindBugs (Khalid et al. 2015), Yasca (Kanawati 2014), and SpongeBugs (Marcilio et al. 2020). The PMD tool was initially open source and was released by DARPA. These widely used traditional static vulnerability tools have reached a certain level of maturity but generally suffer from high FP rates and inefficiency.

Intelligent vulnerability detection has become a research hotspot with the development of AI (Liang et al. 2025). Intelligent vulnerability detection uses machine learning algorithms to automatically identify vulnerabilities in source code through training and learning from a large volume of vulnerabilities samples (Shiri Harzevili et al. 2024). This approach has promoted the performance of vulnerability detection to some extent. So, depending on whether machine learning technologies are employed, software vulnerability detection can be classified into two categories, traditional and intelligent.

6.1 Traditional Vulnerability Detection

Traditional vulnerability detection is broadly classified into three categories, such as similarity, rule, and symbolic execution.

The main idea of the similarity-based vulnerability detection method is that similar code segments may contain similar vulnerabilities. Researchers differentiate these methods based on various ways of representing code features, including text, metrics, tokens, trees, and graphs. For example, Sun et al. (2021) proposed VDSimilar. VDSimilar considers the similarity of vulnerabilities as a key aspect of vulnerability detection. They prepared a relatively small dataset consisting of vulnerabilities and corresponding patches, attempted to identify vulnerability through comparing the similarity between vulnerabilities and the differences between vulnerability patches. When it was published, VDSimilar outperformed the best deep learning-based vulnerability detection method, with its AUC value reaching approximately 97.17% in OpenSSL.

Symbolic execution-based vulnerability detection primarily utilizes advanced symbolic execution analysis technology, which combines intermediate language representation with symbolic execution to solve the constraint in detecting vulnerabilities. For example, Liang et al. (2016) proposed an analysis tool called MLSA. It is a static analysis tool which combines symbolic execution with the Z3-solver using Satisfiability Modulo Theories in vulnerability detection, bases on Low-Level Virtual Machine Intermediate Representation. MLSA is capable of detecting zero error division, pointer overflow, and dead code.

Rule-based vulnerability detection is similar to traditional vulnerability detection tools in that it relies on manual analysis of the types of vulnerabilities and expert rules. The vulnerability rules are used to build a lexical and syntactic parse model of source code, conduct data flow analysis, and analyze taints, and so on. In the Li et al. (2019) survey, there are many common open-source tools such as Flawfinder (Chahar et al. 2012), RATS (Shahriar and Zulkernine 2012) and ITS4. There are also some commercial tools such as Checkmarx (Baset and Denning 2017), Fortify, and Coverity. These tools typically employ simple parsers and vulnerability rules, and result in higher FP and FN rates. Due to the nature of manual rule generation, subjective elements are involved, so the rules may not be comprehensive enough to account for all scenarios. Therefore, despite their relatively high accuracy, these methods still suffer from high FP and FN rates.

We reviewed the main categories of traditional vulnerability detection methods previously. These methods have played an important role in software vulnerability detection. However, they also have limitations and face challenges such as high FP and NP rates and manual incomplete rules. With the development of deep learning technology, researchers have started exploring its application in vulnerability detection to enhance detection performance and reduce false positives.

6.2 Intelligent Vulnerability Detection

Intelligent vulnerability detection methods can be classified into two categories such as traditional and deep learning. The traditional intelligent vulnerability detection method mainly defines the attribute of features manually and uses machine learning technology for classification. It is classified into three types (Napier et al. 2023).

The first type is based on software metrics, uses supervised machine learning, uses known software metrics as feature sets to build predictive models, and then measures other software. For example, Zimmermann et al. (2010) proposed to construct a classification model using metrics in the field of vulnerability prediction to predict vulnerabilities within binary modules in Windows Vista. They used five metrics (code modification, complexity, coverage, dependency, and organization), adopted a binary logistic regression model, and achieved an accuracy of 67%. The advantage of this type of method is that the software metrics are easy to obtain and the technology tends to mature, and it has been applied in some software projects. However, there are three main drawbacks to this type of method: the vulnerability data in the dataset is scarce and sparse, there is a lack of cross-project research, and the reported research results are rather poor. One of the reasons for these problems is that traditional software metrics may not be suitable for the field of vulnerability detection in source code.

The second type is based on anomaly detection. It uses unsupervised learning to extract normal patterns and rules from source codes. Thus, it detects vulnerabilities as abnormal behaviors that deviate from the rules or normal patterns. This type of method often combines data mining techniques to extract code rules. For example, Ant-Miner proposed by Liang et al. (2016) and NAR-Miner proposed by Bian et al. (2018), based on association rules, detected vulnerabilities that were not detected by previous methods. The main limitations of this type of method are as follows: it is mainly applicable to mature software systems; the use of APIs needs to be relatively frequent in the code base; it is unable to specify the types of vulnerabilities; and the false positive rate remains relatively high.

The third type is the identification of vulnerable codes, which uses supervised learning to extract patterns of vulnerable segments from many vulnerable code samples. This type of method requires the collection of large datasets containing software vulnerability samples, the extraction of vulnerability feature vectors from them, and the use of machine learning models to automatically learn software vulnerability patterns. Grieco et al. (2016) tested several machine learning algorithms, such as logistic regression, multilayer perceptron (MLP), and random forest, to detect vulnerabilities in memory corruption in x86 programs. This type of method also has limitations, such as being unable to identify the types of vulnerability and having to obtain information on different aspects of the software.

Unlike traditional machine learning which requires human experts to manually define features, deep learning-based vulnerability detection uses deep neural network models to

automatically learn vulnerability data, which is more automatic and intelligent. This not only helps improve the accuracy rate of software vulnerability detection, but also effectively reduces the false positive rate and false negative rate (Mianxue et al. 2021; Li et al. 2018). Deep learning-based vulnerability detection has achieved remarkable results in programs written in different programming languages. For vulnerabilities in C/C++ programs, Vul-Deepeer (Li et al. 2018) performs vulnerability detection at the slice level. However, it can only detect vulnerabilities related to API calls and utilizes only semantic information caused by data dependencies. SySeVR (Li et al. 2022) introduced syntax/semantic-based vulnerability candidates, and its F1 performance in detecting vulnerabilities in C/C++ programs has reached the state-of-the-art (SOTA). For vulnerabilities in Java programs, Achilles has achieved an accuracy of more than 90% for 24 of the 29 common vulnerabilities in the NIST Java vulnerability dataset, becoming a benchmark method often used for comparison in recent years. In response to disappearance gradient problems when deep neural networks directly learn the information of code ASTs and the destruction of the original syntactic structure of the source code, the ASTGNN method (Zhang et al. 2019) divides code ASTs into smaller statement trees, utilizes a recursive encoder to extract lexical and syntactic information, and uses BiGRU to generate code fragment vectors. Lin and Cai (2021) evaluated the performance of LSTM, BiLSTM, CNN, TCN, GRU, BiGRU, and CNN on the NVD dataset. The results showed that BiLSTM had higher accuracy and precision than LSTM, the performance improvement of BiGRU over GRU was minimal, and CNN had the most balanced performance in terms of accuracy, F1 score, false positive rate, false negative rate, and precision. The AURC (Hu et al. 2023) framework uses a pre-trained classifier and four heuristic rules to detect vulnerabilities and identify defective API calls. In addition, deep learning-based binary code analysis has also achieved some progress recently (Yang et al. 2023; Abu-Mahfouz et al. 2024; Duan and Yin 2024).

6.3 Data Augmentation Techniques

The range of data augmentation techniques varies from simple rule-based methods to more sophisticated generative approaches, which require ensuring the effect of the augmented data. In other words, the use of data augmentation techniques should guarantee generated data in the same distribution as original data in a specific task. For example, the generated data should possess similar semantic features as the original data in machine translation. So, the generated data should have the same labels in the text classification task. Data augmentation technologies are commonly employed in computer vision research. By applying transformations such as rotation, flipping, cropping, and scaling to the original images, multiple new high-quality images of the same category can be generated, effectively improving the training dataset. Data augmentation technologies not only increase the generalization capability of downstream models, but also reduce the sensitivity of specific transformations, thus preventing overfitting. In recent years, data augmentation technologies have also been gradually applied in natural language processing (NLP). Li et al. (2022) have summarized the relative research works of NLP. They classified the works into three categories according to the diversity of generated samples. They also analyzed the advantages and disadvantages of each category. In short, the methods discussed in Li et al. (2022) can generate similar samples in semantics and labels to meet specific task requirements, thus expanding the training dataset and improving the performance of downstream models.

In intelligent software research, source code in the programming language has many similarities with natural language, so researchers have also applied data augmentation technologies to code detection in recent years (Zhang et al. 2021; Dong et al. 2024; Pinku et al. 2023; Ganz et al. 2023). For example, Yedida and Menzies (2022b) used oversampling techniques in the prediction of software vulnerability and achieved favorable results in 14 of 20 datasets in the respective domain. In subsequent studies, the same authors applied data augmentation techniques to code smell detection. They used fuzzy sampling techniques and tested them on a code smell detection dataset (Yedida and Menzies 2022a), obtained better results compared to other complex models, especially on imbalanced datasets.

In conclusion, data-augmentation technologies have proven to be effective in improving model performance and generalization in various domains. By generating additional synthetic training data, data augmentation improves data diversity, improves model robustness, and addresses the challenges of insufficient and unbalanced data. As a result, data augmentation has become an important technology in machine learning and especially deep learning.

7 Conclusion

This paper proposes a novel multiperspective data augmentation method called MPDA. MPDA selectively applies different data augmentation strategies according to vulnerability types and downstream neural networks. Using this method, we effectively address the issues of imbalance and low-quality vulnerability data, provide higher-quality data to train downstream neural networks, and significantly improve deep learning models for vulnerability detection.

In the experiments, we applied our method to four commonly used deep learning models in vulnerability detection and achieve significant improvements, with detection effectiveness increasing by at least 12%. In particular, for the basic LSTM and GRU, our method demonstrated even greater improvements, with an average increase of more than 57%. This validates the effectiveness and feasibility of our approach.

We also recognize that the potential value of our method can be extended to other programming languages. In the future, we plan to refine the strategy selection and expand the application to more programming languages to enhance the method's applicability and generality.

In summary, our research provides a new perspective and approach to data augmentation in vulnerability detection, offers valuable insights to improve the performance of vulnerability detection models, and can also be expanded to other programming languages. We have high expectations for future research and believe that this field will see further development and breakthroughs.

Author Contributions Feiqiao Mao made substantial contributions to the conception and design of the work, critically reviewed the draft and manuscript of the work for important intellectual content. Yingxiang Yuan and Xinyang Du made the acquisition, analysis, interpretation of data, and the creation of new software used in the work, and Yingxiang Yuan drafted the work. Feiqiao Mao and Yingxiang Yuan wrote the main manuscript text, and Xinyang Du prepared tables and figures and the latex sources files of the manuscript. Li Gao did part of the investigation and visualization work. Zhihua Du managed the project, provided resources, funding, and language editing. All authors reviewed and revised the manuscript.

Funding This research was supported by the China National Natural Science Foundation (62176164), the Guangdong Province Natural Science Foundation (Grant 2023A1515010992) and the Shenzhen Science and Technology Foundation (JCYJ20220531101217039 and JCYJ20210324093212034).

Data Availability Statements The authors confirm that the data supporting the findings of this study are available in the paper. Raw data supporting the findings of this study are available from the corresponding author upon reasonable request. We have released the codes and data in this research work along with the annotated data to facilitate further research at <https://github.com/Yfeix/MPDA>.

Declarations

Conflict of Interests The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethical Approval Not applicable.

Informed Consent All datasets used in this study are publicly available from the Open Web Application Security Project (OWASP) community repository on GitHub. In addition, the software and libraries used in this research are free and open source. No human or animal subjects were involved in this study.

Clinical trial number Not applicable.

References

- Abu-Mahfouz A, Alrabaee S, Khasawneh M, Gergely M, Choo K-KR (2024) A deep learning approach to discover router firmware vulnerabilities. *IEEE Trans Industr Inf* 20(1):691–702. <https://doi.org/10.1109/TII.2023.3269774>
- Austin A, Holmgreen C, Williams L (2013) A comparison of the efficiency and effectiveness of vulnerability discovery techniques. *Inf Softw Technol* 55(7):1279–1288
- Baset AZ, Denning T (2017) Ide plugins for detecting input-validation vulnerabilities. In: 2017 IEEE Security and Privacy Workshops (SPW). IEEE, pp 143–146
- Bian P, Liang B, Shi W, Huang J, Cai Y (2018) Nar-miner: discovering negative association rules from code for bug detection. In: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), pp 411–422. <https://doi.org/10.1145/3236024.3236032>
- Byers R, Turner C, Brewer T (2022) National vulnerability database, national institute of standards and technology
- Chahar C, Chauhan VS, Das ML (2012) Code analysis for software and system security using open source tools. *Inf Secur J: A Glob Perspect* 21(6):346–352
- Chawla NV, Bowyer KW, Hall LO, Kegelmeyer WP (2002) Smote: synthetic minority over-sampling technique. *J Artif Intell Res* 16:321–357
- Croft R, Babar MA, Kholoosi MM (2023) Data quality for software vulnerability datasets. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp 121–133. <https://doi.org/10.1109/ICSE48619.2023.00022>
- Dong Z, Hu Q, Zhang Z, Zhao J (2024) On the effectiveness of graph data augmentation for source code learning. *Knowl-Based Syst* 285:111328. <https://doi.org/10.1016/j.knosys.2023.111328>
- Duan LGQY, Yin H (2024) Sigmadiff: semantics-aware deep graph matching for pseudocode diffing. In: Network and Distributed System Security (NDSS) Symposium 2024, pp 1–19. <https://dx.doi.org/10.14722/ndss.2024.23208>
- Du M, Li F, Zheng G, Srikanth V (2017) Deeplog: anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp 1285–1298
- Feng Z, Guo D, Tang D, Duan N, Feng X, Gong M, Shou L, Qin B, Liu T, Jiang D et al (2020) Codebert: a pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*
- Ganz T, Imgrund E, Härterich M, Rieck K (2023) Codegraphsmote-data augmentation for vulnerability discovery. In: Data and applications security and privacy XXXVII, pp 282–301

- Goodfellow I, Pouget-Abadie J, Mirza M, Xu B, Warde-Farley D, Ozair S, Courville A, Bengio Y (2020) Generative adversarial networks. *Commun ACM* 63(11):139–144
- Goodfellow I, Bengio Y, Courville A (2016) Deep learning. MIT Press, ????. <http://www.deeplearningbook.org>
- Grieco G, Grinblat GL, Uzal L, Rawat S, Feist J, Mounier L (2016) Toward large-scale vulnerability discovery using machine learning. In: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy (CODASPY), pp 85–96. <https://doi.org/10.1145/2857705.2857720>
- Han H, Wang W-Y, Mao B-H (2005) Borderline-smote: a new over-sampling method in imbalanced data sets learning. In: International conference on intelligent computing. Springer, pp 878–887
- He H, Bai Y, Garcia EA, Li S (2008) Adasyn: adaptive synthetic sampling approach for imbalanced learning. In: 2008 IEEE international joint conference on neural networks (IEEE world congress on computational intelligence). Ieee, pp 1322–1328
- Hu P, Liang R, Cao Y, Chen K, Zhang R (2023) AURC: detecting errors in program code and documentation. In: 32nd USENIX Security Symposium (USENIX Security 23), pp 1415–1432. <https://www.usenix.org/conference/usenixsecurity23/presentation/hu>
- Kanawati R (2014) Yasca: an ensemble-based approach for community detection in complex networks. In: Computing and combinatorics: 20th international conference, COCOON 2014, Atlanta, GA, USA, August 4–6, 2014. Proceedings 20. Springer, pp 657–666
- Khalid H, Nagappan M, Hassan AE (2015) Examining the relationship between findbugs warnings and app ratings. *IEEE Softw* 33(4):34–39
- Kota VR, Munisamy SD (2022) High accuracy offering attention mechanisms based deep learning approach using cnn/bi-ilstm for sentiment analysis. *Int J Intell Comput Cybern* 15(1):61–74
- Li Z, Zou D, Wang Z, Jin H (2019) Survey on static software vulnerability detection for source code. *Chin J Netw Inf Secur* 5(1):1–14
- Li B, Hou Y, Che W (2022) Data augmentation approaches in natural language processing: a survey. *Ai Open* 3:71–90
- Li Z, Zou D, Xu S, Jin H, Zhu Y, Chen Z (2022) Sysevr: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans Dependable Secure Comput* 19(4):2244–2258
- Liang C, Wei Q, Du J, Wang Y, Jiang Z (2025) Survey of source code vulnerability analysis based on deep learning. *Comput Secur* 148:104098. <https://doi.org/10.1016/j.cose.2024.104098>
- Liang B, Bian P, Zhang Y, Shi W, You W, Cai Y (2016) Antminer: mining more bugs by reducing noise interference. In: 2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE), pp 333–344. <https://doi.org/10.1145/2884781.2884870>
- Liang H, Wang L, Wu D, Xu J (2016) Mlsa: a static bugs analysis tool based on llvm ir. In: 2016 17th IEEE/ACIS international conference on software engineering, artificial intelligence, networking and parallel/distributed computing (SNPD). IEEE, pp 407–412
- Lin W, Cai S (2021) An empirical study on vulnerability detection for source code software based on deep learning. In: 2021 IEEE 21st international conference on software quality, reliability and security companion (QRS-C), pp 1159–1160. <https://doi.org/10.1109/QRS-C55045.2021.00173>
- Li Z, Zou D, Xu S, Ou X, Jin H, Wang S, Deng Z, Zhong Y (2018) Vuldeepecker: a deep learning-based system for vulnerability detection. *arXiv preprint arXiv:1801.01681*
- Luo Y, Xu W, Xu D (2022) Compact abstract graphs for detecting code vulnerability with gnn models. In: Proceedings of the 38th annual computer security applications conference, pp 497–507
- Marcilio D, Furia CA, Bonifácio R, Pinto G (2020) Spongebugs: automatically generating fix suggestions in response to static code analysis warnings. *J Syst Softw* 168:110671
- Mianxue G, Hongyu S, Dan H, Su Y, Wanying C, Zhen G, Chunjie C, Wenjie W, Yuqing Z (2021) Software security vulnerability mining based on deep learning. *Comput Res Dev* 58(10):2140–2162. <https://doi.org/10.7544/issn1000-1239.2021.20210620>
- Mikolov T, Chen K, Corrado G, Dean J (2013) Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*
- Napier K, Bhowmik T, Wang S (2023) An empirical study of text-based machine learning models for vulnerability detection. *Empir Softw Eng* 38:28. <https://doi.org/10.1007/s10664-022-10276-6>
- Partenza G, Amburgey T, Deng L, Dehlinger J, Chakraborty S (2021) Automatic identification of vulnerable code: investigations with an ast-based neural network. In: 2021 IEEE 45th annual computers, software, and applications conference (COMPSAC). IEEE, pp 1475–1482
- Pinku SN, Mondal D, Roy CK (2023) Pathways to leverage transcompiler based data augmentation for cross-language clone detection. In: 2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC), pp 169–180. <https://doi.org/10.1109/ICPC58990.2023.00031>
- Pradhan T, Bhatia C, Kumar P, Pal S (2021) A deep neural architecture based meta-review generation and final decision prediction of a scholarly article. *Neurocomputing* 428:218–238

- Saccante N, Dehlinger J, Deng L, Chakraborty S, Xiong Y (2019) Project achilles: a prototype tool for static method-level vulnerability detection of java source code using a recurrent neural network. In: 2019 34th IEEE/ACM international conference on Automated Software Engineering Workshop (ASEW). IEEE, pp 114–121
- Semasaba AOA, Zheng W, Wu X, Agyemang SA (2020) Literature survey of deep learning-based vulnerability analysis on source code. *IET Softw* 14:654–664. <https://doi.org/10.1049/iet-sen.2020.0084>
- Shahriar H, Zulkernine M (2012) Mitigating program security vulnerabilities: approaches and challenges. *ACM Comput Surv (CSUR)* 44(3):1–46
- Shen G, Tan Q, Zhang H, Zeng P, Xu J (2018) Deep learning with gated recurrent unit networks for financial sequence predictions. *Procedia Comput Sci* 131:895–903
- Shiri Harzevili N, Boaye Belle A, Wang J, Wang S, Jiang ZMJ, Nagappan N (2024) A systematic literature review on automated software vulnerability detection using machine learning. *ACM Comput Surv* 57(3)
- Sun H, Cui L, Li L, Ding Z, Hao Z, Cui J, Liu P (2021) Vdsimilar: vulnerability detection based on code similarity of vulnerabilities and patches. *Comput Secur* 110:102417
- Sun C, Shrivastava A, Singh S, Gupta A (2017) Revisiting unreasonable effectiveness of data in deep learning era. In: 2017 IEEE International Conference on Computer Vision (ICCV), pp 843–852. <https://doi.org/10.1109/ICCV.2017.97>
- Tang Z, Hu Q, Hu Y, Kuang W, Chen J (2022) Sevuldet: a semantics-enhanced learnable vulnerability detector. In: 2022 52nd Annual IEEE/IFIP international conference on Dependable Systems and Networks (DSN), pp 150–162
- Wang H, Ye G, Tang Z, Tan SH, Huang S, Fang D, Feng Y, Bian L, Wang Z (2020) Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans Inf Forensics Secur* 16:1943–1958
- Wang M, Tao C, Guo H (2023) Lcvd: loop-oriented code vulnerability detection via graph neural network. *J Syst Softw* 202:111706. <https://doi.org/10.1016/j.jss.2023.111706>
- Yang S, Dong C, Xiao Y, Cheng Y, Shi Z, Li Z, Sun L (2023) Asteria-pro: enhancing deep learning-based binary code similarity detection by incorporating domain knowledge. *ACM Trans Softw Eng Methodol* 33(1). <https://doi.org/10.1145/3604611>
- Yedida R, Menzies T (2022b) On the value of oversampling for deep learning in software defect prediction. *IEEE Trans Software Eng* 48(8):3103–3116. <https://doi.org/10.1109/TSE.2021.3079841>
- Yedida R, Menzies T (2022a) How to improve deep learning for software analytics: (a case study with code smell detection). In: Proceedings of the 19th international conference on mining software repositories, pp 156–166
- Zhang D, Tian L, Hong M, Han F, Ren Y, Chen Y (2018) Combining convolution neural network and bidirectional gated recurrent unit for sentence semantic classification. *IEEE Access* 6:73750–73759
- Zhang H, Bi Y, Guo H, Sun W, Li J (2021) Isvsf: intelligent vulnerability detection against java via sentence-level pattern exploring. *IEEE Syst J* 16(1):1032–1043
- Zhang J, Wang X, Zhang H, Sun H, Wang K, Liu X (2019) A novel neural source code representation based on abstract syntax tree. In: 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), pp 783–794. <https://doi.org/10.1109/ICSE.2019.00086>
- Zhang X, Zhou Y, Han T, Chen T (2021) Training deep code comment generation models via data augmentation. In: Proceedings of the 12th Asia-Pacific symposium on internetware, pp 185–188. <https://doi.org/10.1145/3457913.3457937>
- Zimmermann T, Nagappan N, Williams L (2010) Searching for a needle in a haystack: predicting security vulnerabilities for windows vista. In: 2010 third international conference on software testing, verification and validation, pp 421–428. <https://doi.org/10.1109/ICST.2010.32>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Feiqiao Mao¹  · Yingxiang Yuan¹  · Xingyang Du¹  · Li Gao¹  · Zhihua Du¹ 

✉ Feiqiao Mao
feiqiao@szu.edu.cn

Yingxiang Yuan
1093805529@qq.com

Xingyang Du
duxingyang2023@email.szu.edu.cn

Li Gao
gaoli330@163.com

Zhihua Du
duzh@szu.edu.cn

¹ College of Computer Science and Software Engineering, Shenzhen University, ShenZhen, China